DISPENSARY MANAGEMENT SYSTEM

Database Design and System Architecture

Release 1.0 - modular monolith (Next.js, Spring Boot, PostgreSQL)

Document	02-Database-and-System-Architecture.pdf
Version	1.0.0
Prepared	12 October 2026
Status	Draft for owner review - implementation baseline for release 1.0
Purpose	Defines the architecture, the PostgreSQL schema and the concurrency design that the implementation must follow.

Document set

File	Content
01-Software-Requirements-Specification.pdf	What the system must do: scope, roles, FR/NFR, business rules, workflows, use cases, validation, security, audit, open questions, traceability.
02-Database-and-System-Architecture.pdf	How it is built: architecture, components, sequences, data model, data dictionary, DDL, concurrency, deployment.
03-REST-API-Documentation.pdf	Every endpoint with roles, parameters, examples, validation, errors and transactions (generated from openapi.yaml).
openapi.yaml	Machine-readable OpenAPI 3.1 contract - the source of truth for the API.
backend-requirements.md / frontend-requirements.md	Implementation instructions for Claude Code.

All documents in the set are generated from the same sources (database migration, openapi.yaml and requirement catalogues) and are cross-checked automatically; identifiers such as FR-012, BR-Q-03 or ERR codes are stable across documents.

Contents

1. Introduction 3
1.1 Purpose and scope 3
1.2 Architectural drivers 3
1.3 Key decisions 3

2. System architecture 5
2.1 Architectural style and modules 5
2.2 High-level architecture 6
2.3 Component responsibilities 7
2.4 Backend component diagram 8
2.5 Sequence diagrams 9
2.6 Real-time update strategy 15
2.7 Authentication and authorisation flow 16
2.8 Deployment architecture 17

3. Data model 23
3.1 Identifier strategy 23
3.2 Table inventory - why each table exists 23
3.3 Entity relationship diagrams 24
3.4 Primary and foreign key design 29
3.5 Normalisation decisions 29
3.6 Date of birth and age 30
3.7 Medical history representation 30
3.8 Indexing strategy 30

4. Data dictionary 32

5. SQL DDL 53

6. Example queries 62

7. Transactions and concurrency 65
7.1 Principles 65
7.2 Transaction boundaries and locking per operation 65
7.3 Queue number generation (gap-free) 66
7.4 Call next (compare-and-set) 67
7.5 Idempotency mechanism 67
7.6 Verification performed for this document 67

8. Flyway migration strategy 68

9. Development seed data 69

10. Security architecture summary 72

11. Operations 73

12. Scalability and evolution 74

1. Introduction

1.1 Purpose and scope

This document defines the architecture and the database design of release 1.0 of the Dispensary Management System. It implements the requirements of *01-Software-Requirements-Specification.pdf* (referenced as SRS) and is the design basis for `backend-requirements.md`, `frontend-requirements.md` and `openapi.yaml`. The data dictionary (section 4), DDL (section 5) and ER diagrams (section 3) are generated from the Flyway migration `V1__initial_schema.sql`, which was executed against PostgreSQL 16 while preparing this document.

1.2 Architectural drivers

Driver	Source	Architectural response
Patient data privacy	SRS 12, NFR-010..012	Server-side RBAC, dedicated DTOs per audience, no PHI in logs or browser storage, append-only audit
No duplicate / skipped queue numbers	FR-036, FR-039	Row lock on the queue session + gap-free counter + compare-and-set call-next + unique constraints
Exactly-once dispensing	FR-029	Row lock + state check + `UNIQUE(prescription_id)` + idempotency keys
Atomic clinical records	FR-020	One transaction per command, constraints in the database
Small team, single site	A-01, A-05	Modular monolith, one database, Docker Compose
Near-real-time queue display	FR-044, NFR-004	ETag polling every 5 s
Future growth	SRS 16	Package-per-module boundaries, stateless API, versioned API path, Flyway

1.3 Key decisions

ID	Decision	Rationale	Alternatives rejected
ADR-01	Modular monolith (one Spring Boot deployable, one Next.js app)	Lowest operational cost, simple transactions across modules	Microservices: distributed transactions for consultation + prescription, more infrastructure
ADR-02	UUID primary keys + separate human numbers	Non-enumerable ids in URLs; app-side generation; human numbers stay short	BIGSERIAL ids (enumerable, leak volume)
ADR-03	JWT access token (memory) + rotating refresh cookie	Stateless API, Swagger-friendly, revocation within 15 min, cookie not readable by JS	Server sessions (sticky state), tokens in localStorage (XSS exposure)
ADR-04	Same-origin deployment (reverse proxy / Next.js proxy)	No CORS for browsers, SameSite=Strict cookie works	Cross-origin SPA with credentials
ADR-05	Polling with ETag for queue updates	Works through every proxy, stateless, trivial load at clinic scale	WebSocket, SSE (section 2.6)
ADR-06	Pessimistic row lock on `queue_sessions` for all queue mutations	Simple, deadlock-free serialisation per queue; gap-free numbers	Sequences (gaps on rollback, one per session), SERIALIZABLE retries

ID	Decision	Rationale	Alternatives rejected
ADR-07	Idempotency keys stored in the business transaction	Correct under concurrency without a separate in-flight state; no PHI duplicated	Response caching in Redis
ADR-08	RFC 9457 Problem Details with stable `code`	Standard, native Spring support	Custom envelopes
ADR-09	VARCHAR + CHECK for enums	Easy to evolve in Flyway; maps to JPA `EnumType.STRING`	PostgreSQL ENUM types (ALTER TYPE friction)
ADR-10	Medicine catalogue without stock	Needed for consistent names and "unavailable" (inactive) flag; inventory out of scope	Free text only (inconsistent names), full inventory (out of scope)

2. System architecture

2.1 Architectural style and modules

The backend is one Spring Boot application split into feature packages that own their entities, repositories, services, DTOs and controllers. Cross-module calls go through service interfaces, never through another module's repository.

Module (package)	Responsibility	Depends on
`common`	Problem Details, ErrorCode, paging, clock, idempotency, request id	-
`security`	Filter chain, JWT encode/decode, rate limiting, password-change gate, current-user resolver	common, user
`auth`	Login, refresh, logout, change password, refresh-token store	security, user, audit
`user`	Staff accounts, roles, staff directory, bootstrap admin	audit
`patient`	Registration, lookup, search, updates, medical history read model	consultation (read), prescription (read), audit
`consultation`	Consultations and diagnoses, consultation commands	patient, prescription, queue (read), medicine, audit
`prescription`	Prescriptions, items, state machine, cancel, re-issue, listing	medicine, audit
`pharmacy`	Start and dispense, dispensing records	prescription, audit
`medicine`	Medicine catalogue	audit
`queue`	Sessions, bookings/entries, call-next, transitions, public queue read model	user (read), audit
`audit`	Audit event writer and query	common
`retention`	Scheduled housekeeping jobs	queue, auth, common, audit

2.2 High-level architecture

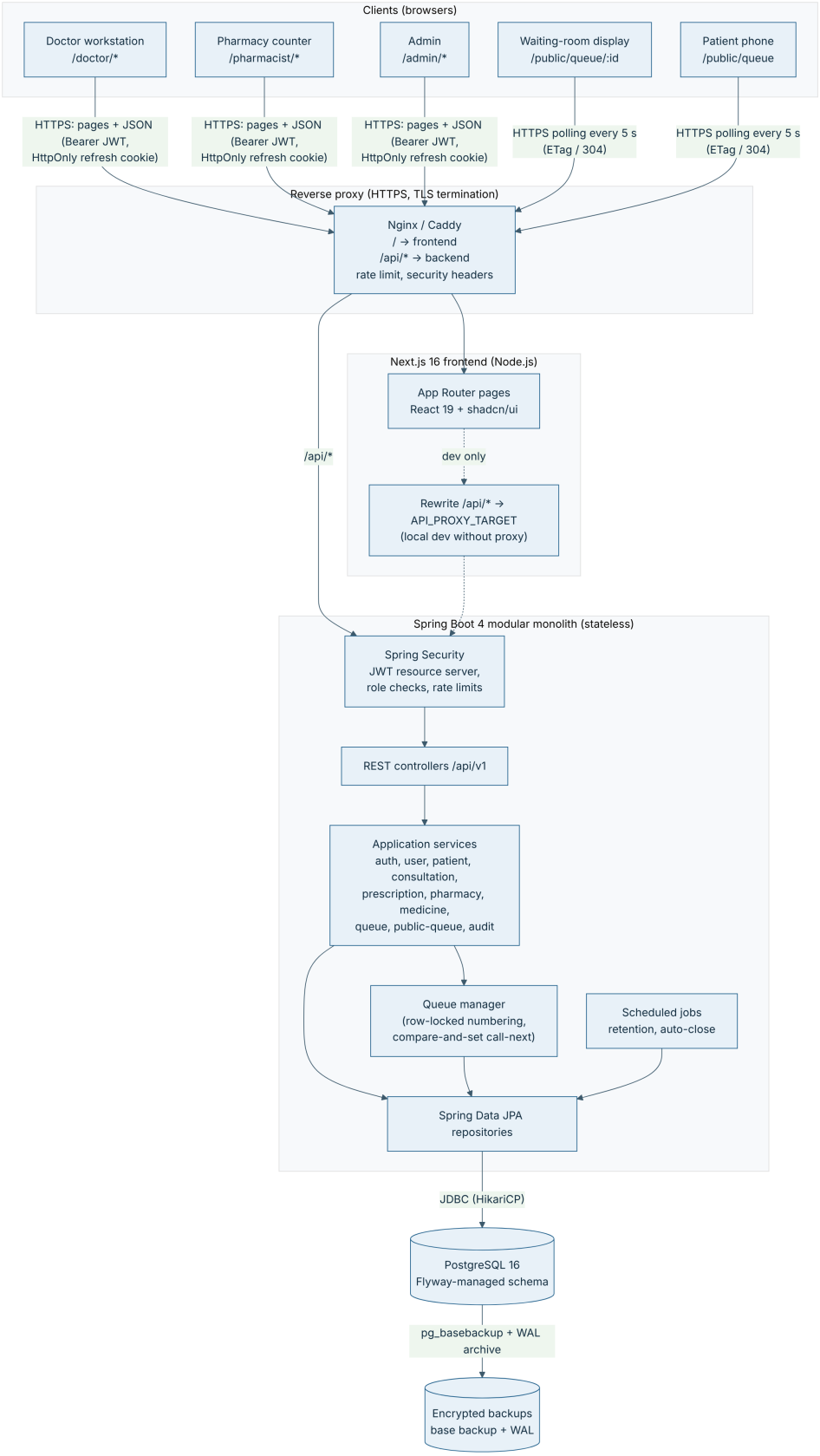


Figure 1. High-level system architecture (source: diagrams/system-architecture.mmd)

2.3 Component responsibilities

Component	Responsibilities	Must not
Next.js frontend	Renders staff dashboards and public pages; client-side form validation (Zod) mirroring the API; TanStack Query cache and polling; holds the access token in memory; route guards for UX; runtime proxy of `/api/*` to the backend when no reverse proxy is present; security headers (CSP with nonce).	Store PHI in browser storage; treat hidden buttons as authorisation; expose secrets via `NEXT_PUBLIC_*`; call the database.
Spring Boot REST API	Implements `openapi.yaml` under `/api/v1`; request validation (Bean Validation); maps exceptions to Problem Details; exposes Actuator health.	Contain business logic in controllers; return JPA entities.
Spring Security	Bearer-JWT authentication (resource server with HMAC key), role mapping from the `roles` claim, method security (`@PreAuthorize`), password-change gate, CORS (only if configured), rate limiting filters, security headers.	Keep HTTP sessions (stateless); log credentials.
Application services	Own transactions (`@Transactional`), enforce business rules and state machines, ownership checks, idempotency, audit writes, call repositories.	Depend on web types (HttpServletRequest etc.).
Repository layer	Spring Data JPA repositories; locking finders (`@Lock(PESSIMISTIC_WRITE)`); projection and batch queries (no N+1).	Contain business decisions.
PostgreSQL	Source of truth; enforces integrity (PK, FK, UNIQUE, CHECK, partial unique indexes, append-only trigger); row locks for serialisation.	Be reachable from outside the private network.
Queue management component (`queue` module)	Numbering (`last_issued_number` under row lock), call-next compare-and-set, entry state machine, session lifecycle, auto-close, public projection.	Use in-memory counters or caches as the source of truth.
Update mechanism (polling)	Public and staff queue views poll `GET` endpoints every 5 s with `If-None-Match`; `queue_sessions.version` is the ETag; unchanged queues cost one indexed primary-key read and return 304.	Push personal data; hold connections open.

2.4 Backend component diagram

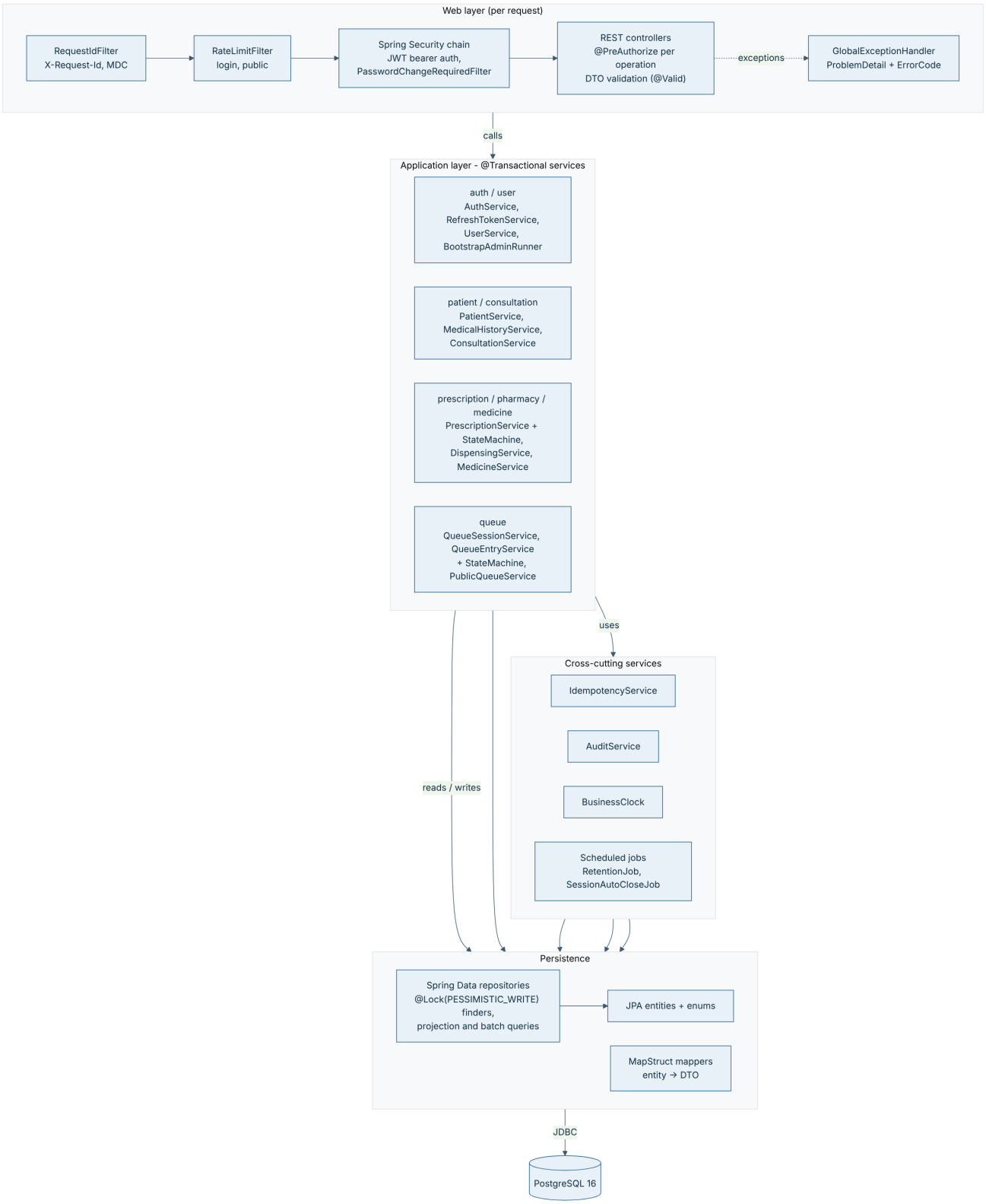


Figure 2. Backend components and layering (source: diagrams/backend-components.mmd)

Layering rules (enforced by ArchUnit tests): controllers → services → repositories; DTOs and mappers live in the module's `dto` / `mapper` packages; entities are never returned from controllers; a module may call another module's *service*, never its repository; `common` depends on nothing.

2.5 Sequence diagrams

Long sequences are printed in parts so that the text stays readable; the parts are generated from the single source file named in each caption (numbering continues across parts).

2.5.1 Consultation to pharmacy (Workflows A and B)

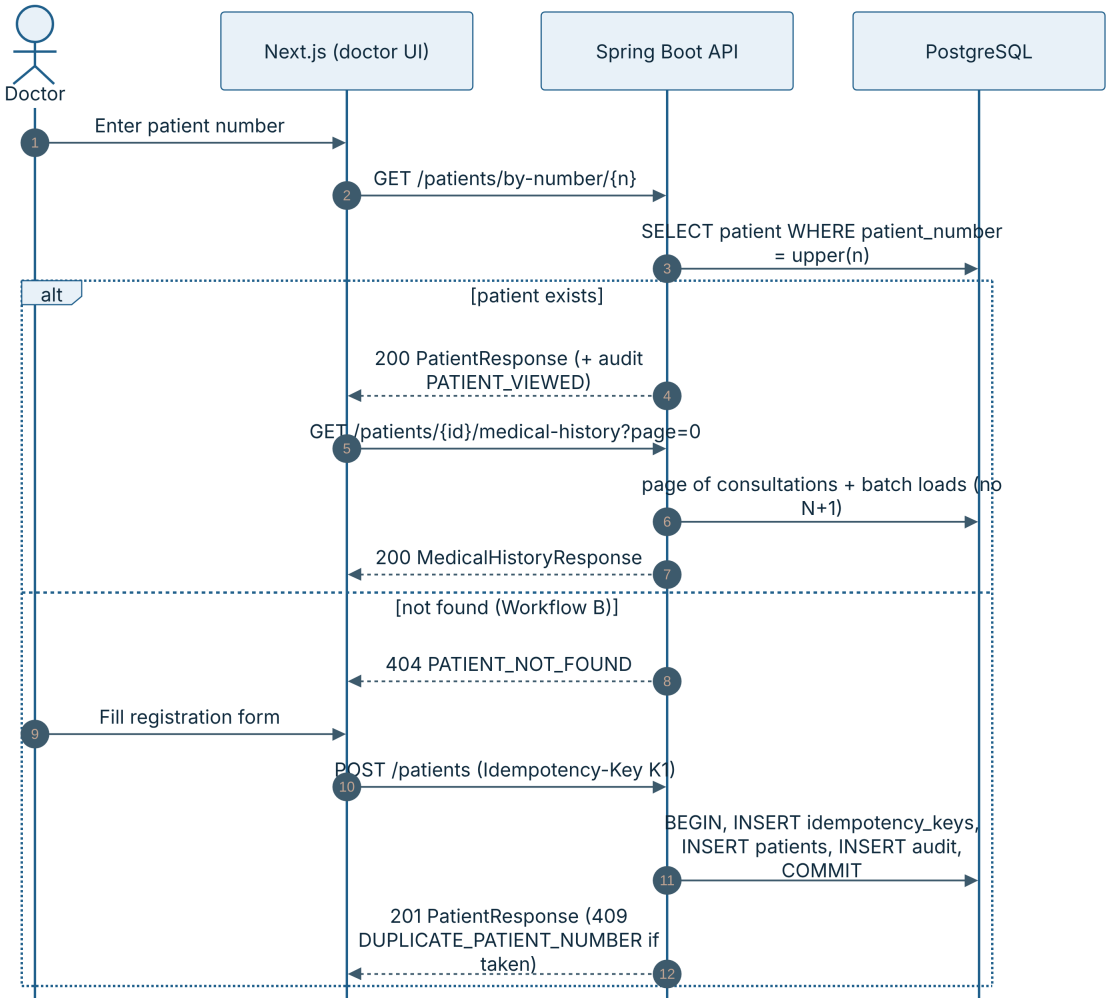


Figure 3. Consultation to pharmacy, part 1 - patient lookup or registration (source file covers all parts) (source: diagrams/consultation-pharmacy-sequence.mmd)

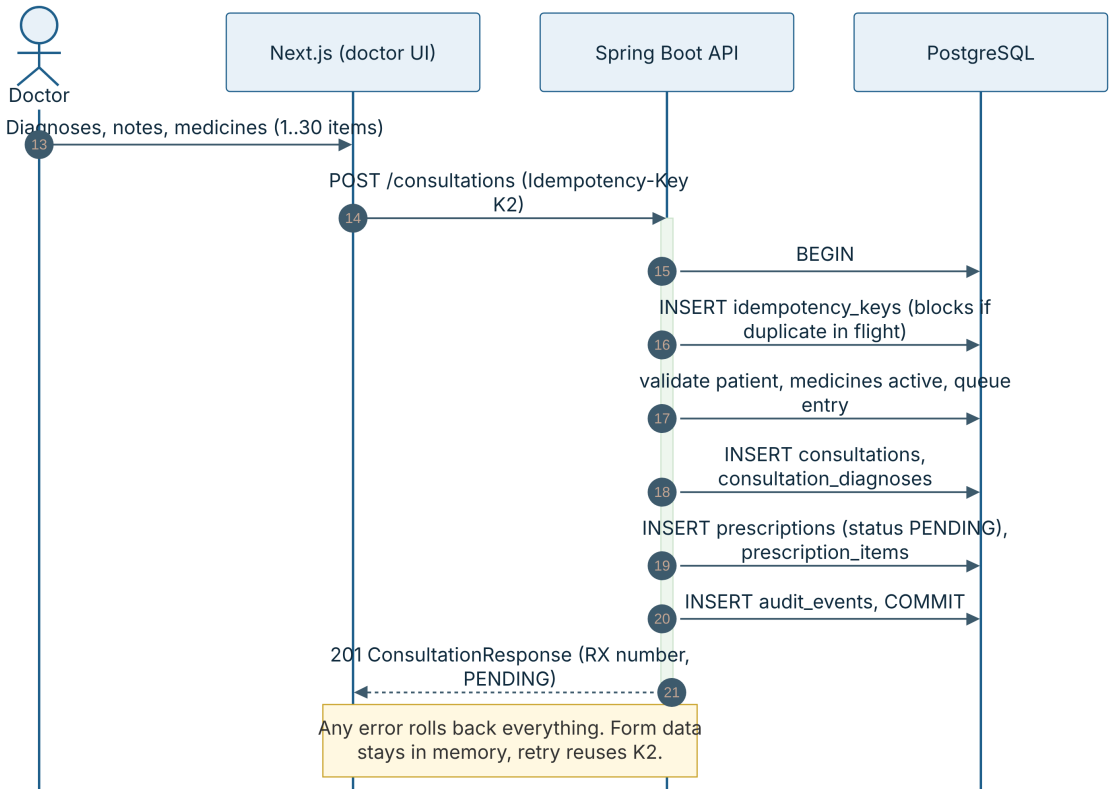


Figure 4. Consultation to pharmacy, part 2 - atomic consultation and prescription submission (source: diagrams/consultation-pharmacy-sequence.mmd)

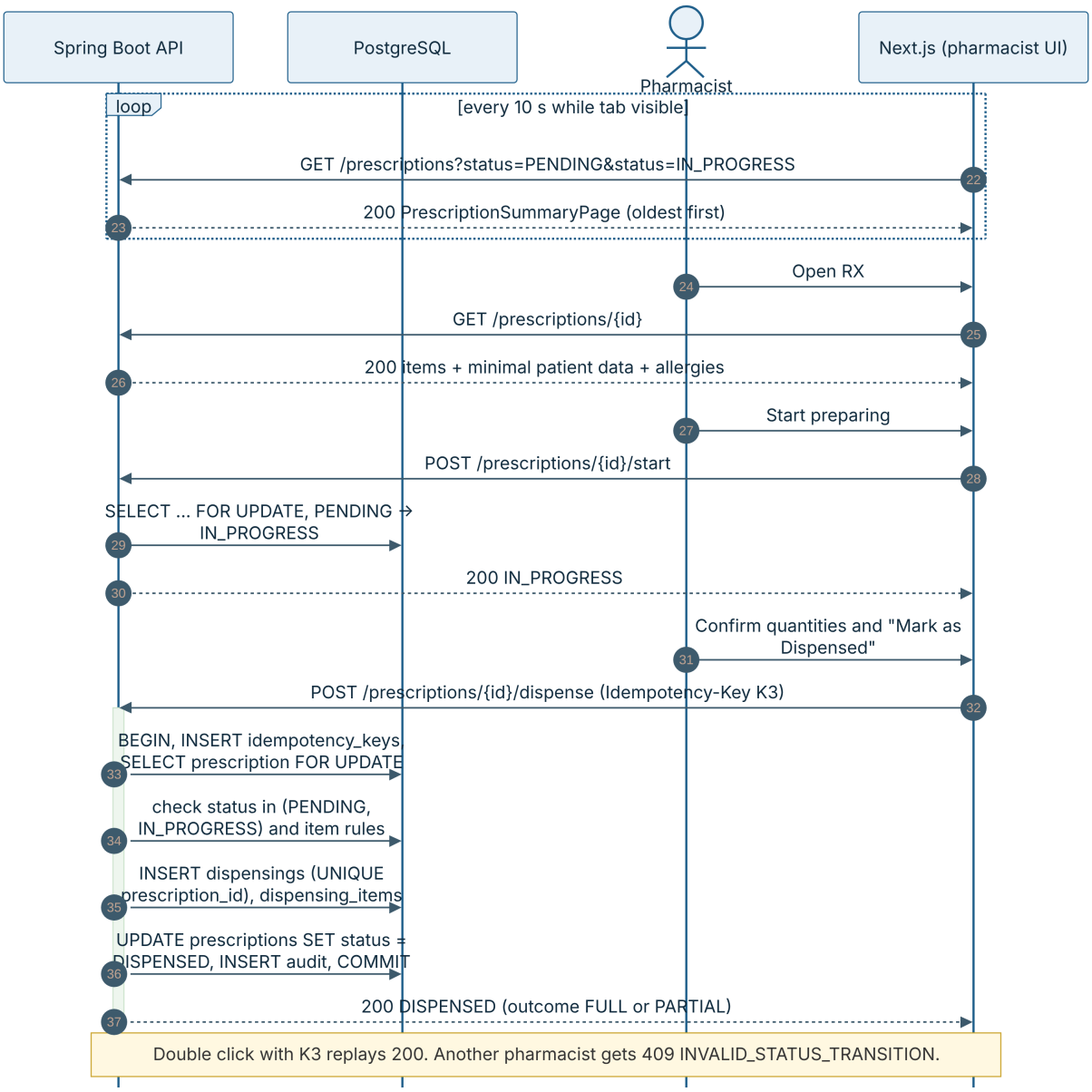


Figure 5. Consultation to pharmacy, part 3 - pharmacist queue, start and dispense (source: diagrams/consultation-pharmacy-sequence.mmd)

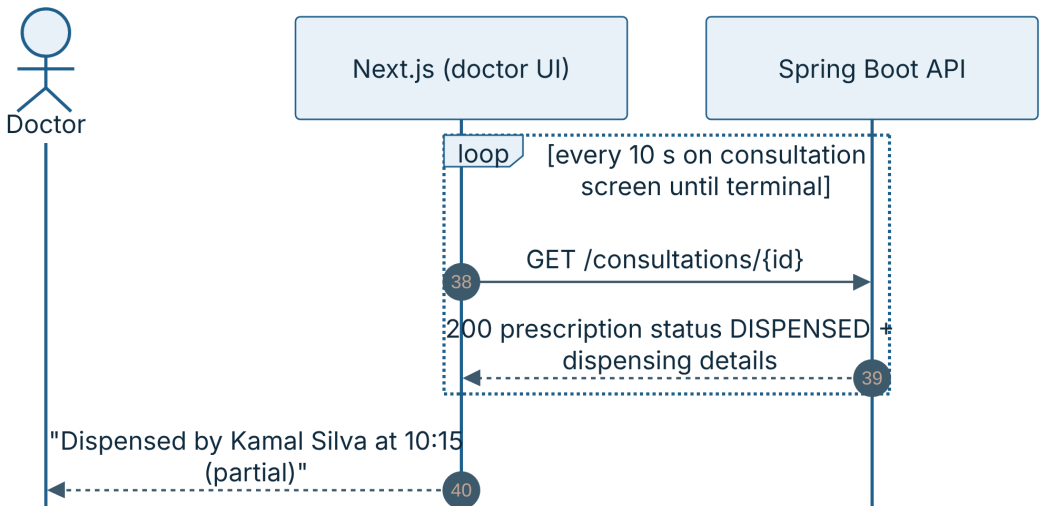


Figure 6. Consultation to pharmacy, part 4 - doctor sees the dispensing status (source: diagrams/consultation-pharmacy-sequence.mmd)

2.5.2 Telephone booking and calling the next patient (Workflows C and E)

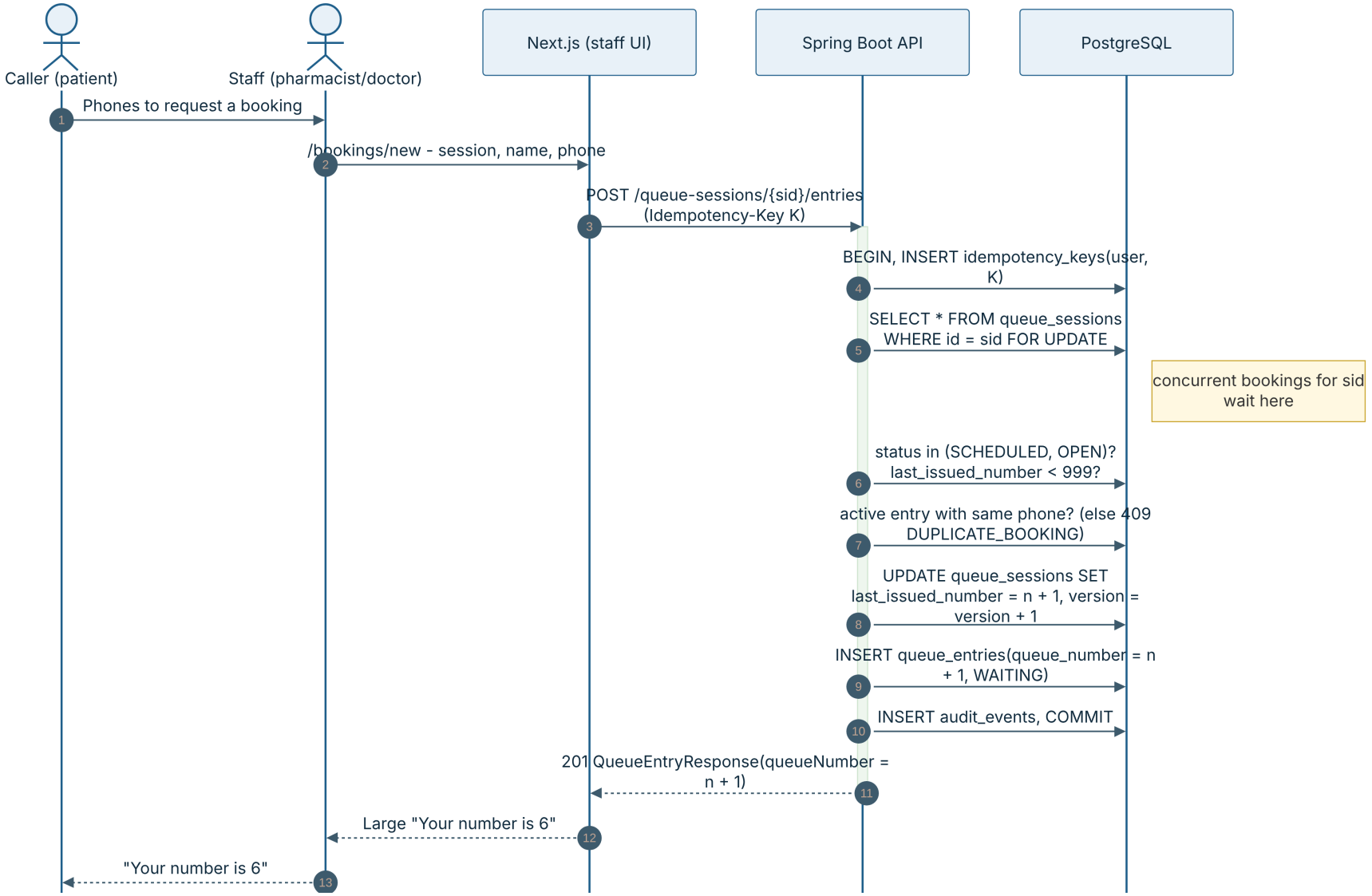


Figure 7. Telephone booking - gap-free number assignment (Workflow C) (source: diagrams/booking-queue-sequence.mmd)

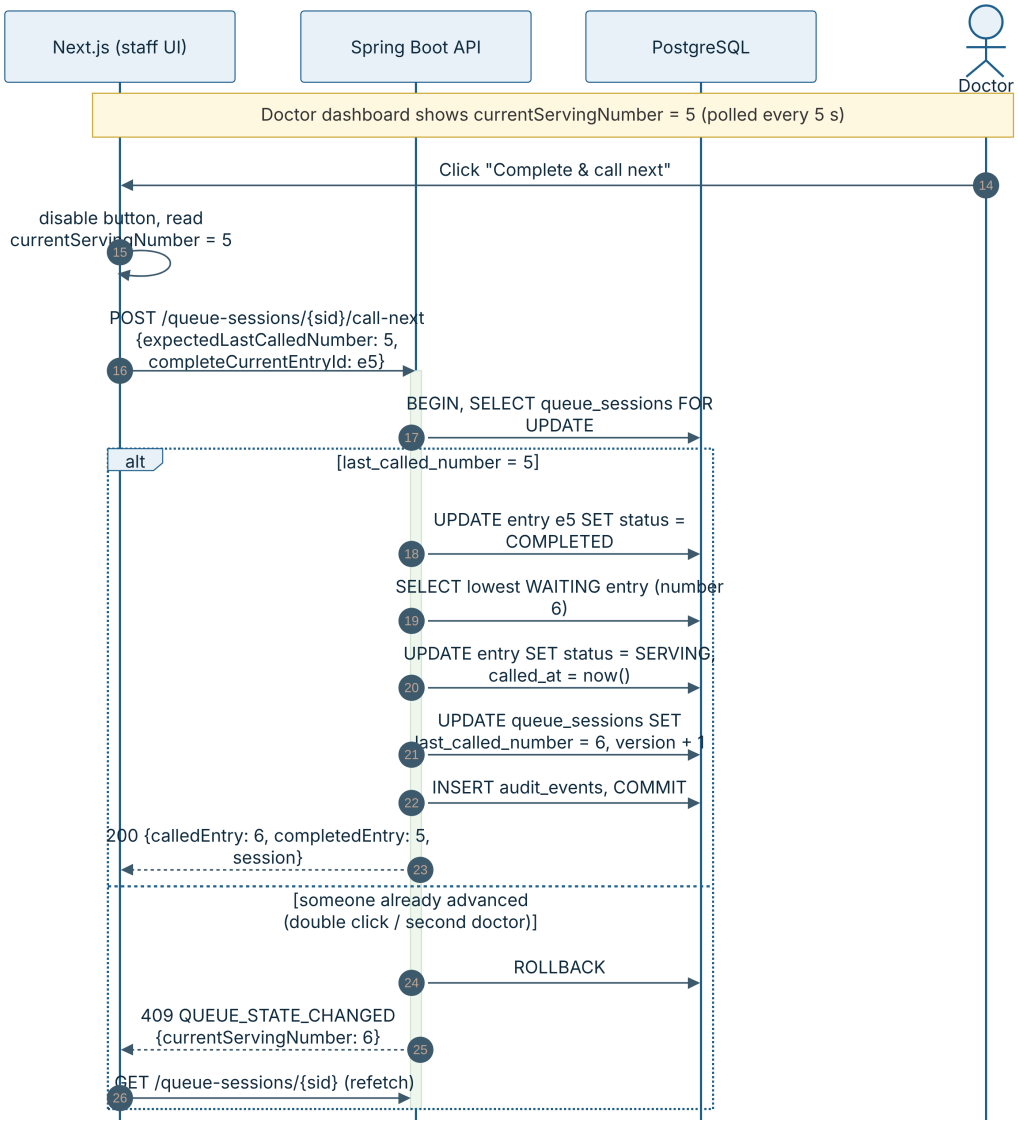


Figure 8. Calling the next patient - compare-and-set (Workflow E) (source: diagrams/booking-queue-sequence.mmd)

2.5.3 Public queue update (Workflow D)

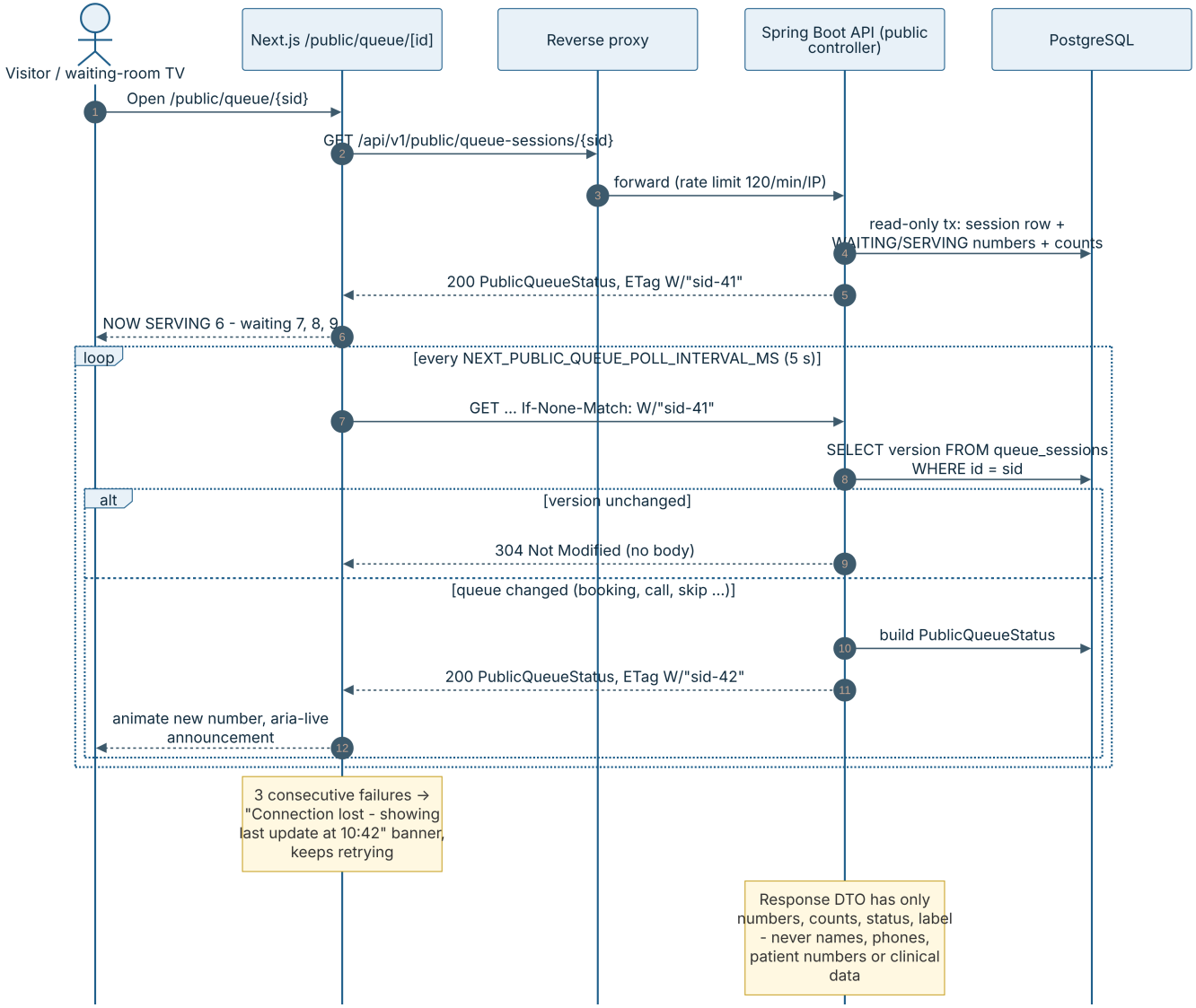


Figure 9. Public queue update sequence with ETag polling (source: diagrams/public-queue-sequence.mmd)

2.6 Real-time update strategy

Criterion	Polling + ETag (chosen)	Server-Sent Events	WebSocket
Latency	<= poll interval (5 s) - meets NFR-004 (7 s)	< 1 s	< 1 s
Server state	none	one open connection per viewer; fan-out needs LISTEN/NOTIFY or a broker when scaled to 2 instances	same as SSE, plus bidirectional protocol
Proxies / networks	works everywhere	needs proxy buffering disabled and long timeouts	needs Upgrade support
Auth for staff views	normal bearer request	EventSource cannot send headers - needs cookie or token-in-URL	custom handshake auth
Implementation and test effort	lowest (plain GET)	medium	highest
Load at clinic scale (250 viewers / 5 s)	50 req/s, mostly 304 from a PK lookup	250 idle connections	250 idle connections

Decision: polling with conditional requests for release 1.0. Rationale: the latency requirement is seconds, not milliseconds; it is stateless (NFR-023), robust on clinic networks and TVs, and needs no extra infrastructure. The public DTO and ETag design allow a later switch to SSE (`/api/v1/public/queue-sessions/{id}/stream` emitting the same `PublicQueueStatus`) without changing clients' rendering code.

2.7 Authentication and authorisation flow

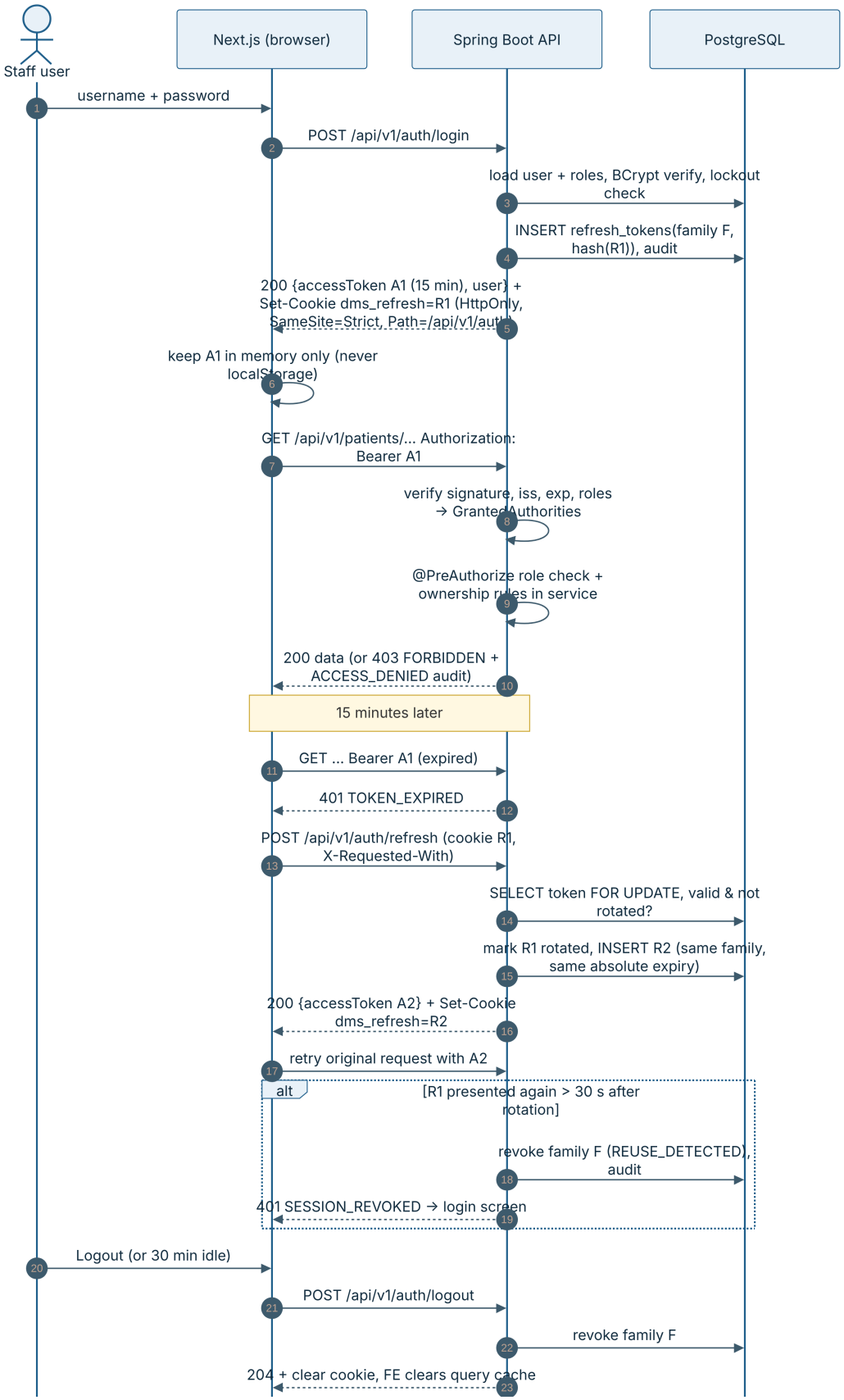


Figure 10. Authentication, token refresh and authorisation (source: diagrams/auth-flow-sequence.mmd)

1. `POST /auth/login` verifies the BCrypt hash (constant-time, dummy hash for unknown users), applies the lockout rules and returns a JWT (`sub`, `roles`, `iss`, `iat`, `exp`, `jti`; 15 min) plus the `dms_refresh` cookie (256-bit random, stored as SHA-256, 12 h family).
2. Every API request carries `Authorization: Bearer`. Spring Security validates signature, issuer and expiry and maps `roles` to `ROLE_DOCTOR` etc. The password-change gate rejects other endpoints while `mustChangePassword` (claim `pwd_change=true`) is set.
3. `@PreAuthorize` on each controller method enforces the roles of `openapi.yaml`; services enforce ownership (e.g. prescriber) and log `ACCESS_DENIED`.
4. On `401 TOKEN_EXPIRED` the browser calls `POST /auth/refresh` once (single-flight), receives a new token and retries.
5. Logout and idle timeout revoke the family and clear the cookie and the client cache.

2.8 Deployment architecture

2.8.1 Local development

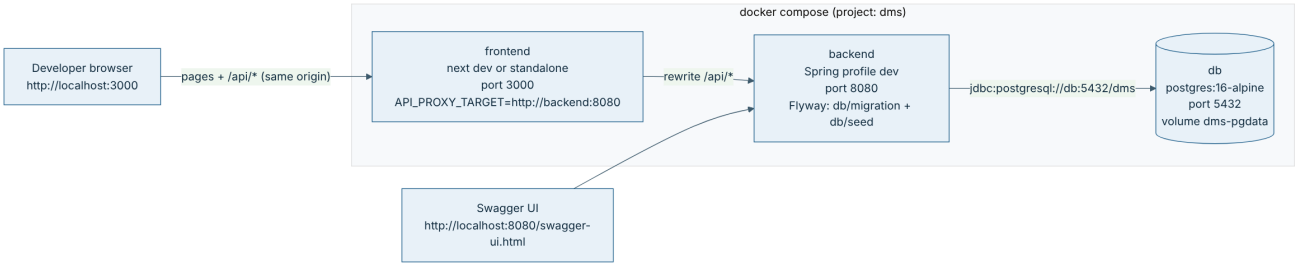


Figure 11. Local development with Docker Compose (source: diagrams/deployment-local.mmd)

The repository layout is a mono-repo: `backend/` (Spring Boot), `frontend/` (Next.js), `docs/` (this document set), `docker-compose.yml` and `.env.example` at the root. `docker compose up --build` starts PostgreSQL, the backend (profile `dev`, migrations + seed) and the frontend.

```yaml
# docker-compose.yml - LOCAL DEVELOPMENT ONLY (repository root)
# Usage: cp .env.example .env && docker compose up --build
name: dms
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"            # remove in shared environments
    volumes:
      - dms-pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 20

  backend:
    build: ./backend
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_PROFILES_ACTIVE: ${SPRING_PROFILES_ACTIVE}
      SERVER_PORT: ${SERVER_PORT}
      DB_URL: ${DB_URL}
      DB_USERNAME: ${DB_USERNAME}
      DB_PASSWORD: ${DB_PASSWORD}
      DB_POOL_MAX_SIZE: ${DB_POOL_MAX_SIZE}
      APP_JWT_SECRET: ${APP_JWT_SECRET}
```

```
      APP_JWT_ISSUER: ${APP_JWT_ISSUER}
      APP_JWT_ACCESS_TOKEN_TTL: ${APP_JWT_ACCESS_TOKEN_TTL}
      APP_JWT_REFRESH_TOKEN_TTL: ${APP_JWT_REFRESH_TOKEN_TTL}
      APP_REFRESH_COOKIE_SECURE: ${APP_REFRESH_COOKIE_SECURE}
      APP_CORS_ALLOWED_ORIGINS: ${APP_CORS_ALLOWED_ORIGINS}
      APP_TIMEZONE: ${APP_TIMEZONE}
      APP_SWAGGER_ENABLED: ${APP_SWAGGER_ENABLED}
      APP_RATE_LIMIT_LOGIN_PER_MINUTE: ${APP_RATE_LIMIT_LOGIN_PER_MINUTE}
      APP_RATE_LIMIT_PUBLIC_PER_MINUTE: ${APP_RATE_LIMIT_PUBLIC_PER_MINUTE}
      APP_RETENTION_QUEUE_PII_DAYS: ${APP_RETENTION_QUEUE_PII_DAYS}
      SERVER_FORWARD_HEADERS_STRATEGY: ${SERVER_FORWARD_HEADERS_STRATEGY}
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health/readiness || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 30

  frontend:
    build:
      context: ./frontend
      args:
        NEXT_PUBLIC_API_BASE_PATH: ${NEXT_PUBLIC_API_BASE_PATH}
        NEXT_PUBLIC_APP_NAME: ${NEXT_PUBLIC_APP_NAME}
        NEXT_PUBLIC_QUEUE_POLL_INTERVAL_MS: ${NEXT_PUBLIC_QUEUE_POLL_INTERVAL_MS}
        NEXT_PUBLIC_PRESCRIPTION_POLL_INTERVAL_MS: ${NEXT_PUBLIC_PRESCRIPTION_POLL_INTERVAL_MS}
        NEXT_PUBLIC_IDLE_TIMEOUT_MINUTES: ${NEXT_PUBLIC_IDLE_TIMEOUT_MINUTES}
    depends_on:
      backend:
        condition: service_healthy
    environment:
      API_PROXY_TARGET: http://backend:8080   # server-side only; overrides the host value in .env
    ports:
      - "3000:3000"

volumes:
  dms-pgdata:
```

2.8.2 Production (reference)

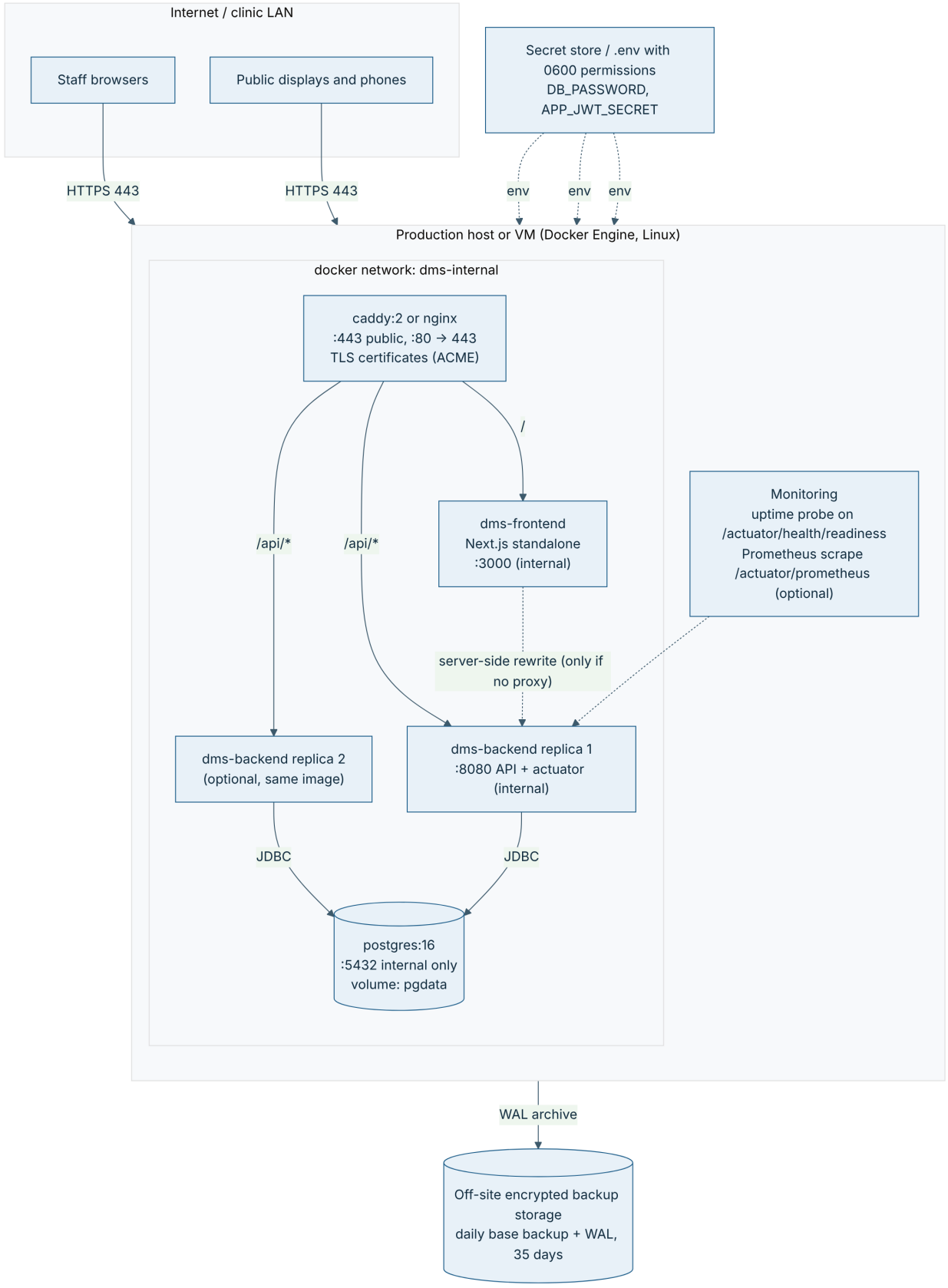


Figure 12. Production deployment (reference) (source: diagrams/deployment-architecture.mmd)

- One Linux VM or host with Docker Engine; a reverse proxy (Caddy or Nginx) terminates TLS and routes `/` to the frontend and `/api/` to the backend; only ports 80/443 are public.

- Backend replicas are stateless; a second replica is optional. `/actuator/prometheus` answers only private-network addresses and is not routed by the proxy.
- PostgreSQL on a private network (or a managed PostgreSQL service) with WAL archiving and daily base backups to encrypted off-site storage (NFR-006).
- Secrets in a secret store or a root-only `.env` file; `SPRING_PROFILES_ACTIVE=prod`; `APP_SWAGGER_ENABLED=false`; `APP_REFRESH_COOKIE_SECURE=true`; `SERVER_FORWARD_HEADERS_STRATEGY=native` only behind the trusted proxy.
- Production hardening of database roles: `dms_owner` owns the schema and runs Flyway (`DB_MIGRATION_USERNAME`); `dms_app` (`DB_USERNAME`) has `SELECT, INSERT, UPDATE, DELETE` on business tables and only `SELECT, INSERT` on `audit_events`; no `TRUNCATE`.

2.8.3 Environment variables

Variable	Component	Required	Secret	Default	Local dev value	Description
`SPRING_PROFILES_ACTIVE`	backend	no	no	`prod`	`dev`	Active Spring profile. dev adds seed data (classpath:db/seed), enables Swagger UI and relaxes cookie Secure flag. Never run dev in production.
`SERVER_PORT`	backend	no	no	`8080`	`8080`	HTTP port of the API.
`DB_URL`	backend	yes	no	-	`jdbc:postgresql://db:5432/dms`	JDBC URL of the PostgreSQL database. Mapped to spring.datasource.url and spring.flyway.url.
`DB_USERNAME`	backend	yes	no	-	`dms`	Database user used by the application and Flyway.
`DB_PASSWORD`	backend	yes	yes	-	`dms_dev_password`	Database password.
`DB_MIGRATION_USERNAME`	backend	no	no	`${DB_USERNAME}`	-	Optional separate schema-owner user for Flyway (production hardening). Defaults to DB_USERNAME.
`DB_MIGRATION_PASSWORD`	backend	no	yes	`${DB_PASSWORD}`	-	Password of DB_MIGRATION_USERNAME. Defaults to DB_PASSWORD.
`DB_POOL_MAX_SIZE`	backend	no	no	`10`	`10`	HikariCP maximum pool size.
`APP_JWT_SECRET`	backend	yes	yes	-	`ZGV2LW9ubHktc2VjcmV0LWNoYW5nZS1tZS0wMTIzNDU2Nzg5YWJjZGVm`	Base64-encoded HMAC-SHA256 key, at least 32 bytes after decoding. Startup fails if missing or shorter. Generate with: openssl rand -base64 48.
`APP_JWT_ISSUER`	backend	no	no	`dms-api`	`dms-api`	iss claim written to and required on access tokens.
`APP_JWT_ACCESS_TOKEN_TTL`	backend	no	no	`PT15M`	`PT15M`	Access-token lifetime (ISO-8601 duration).
`APP_JWT_REFRESH_TOKEN_TTL`	backend	no	no	`PT12H`	`PT12H`	Absolute lifetime of a login (refresh-token family), ISO-8601 duration.
`APP_REFRESH_COOKIE_SECURE`	backend	no	no	`true`	`false`	Secure attribute of the dms_refresh cookie. Must be true in any environment served over HTTPS.
`APP_CORS_ALLOWED_ORIGINS`	backend	no	no	-	`http://localhost:3000`	Comma-separated list of allowed browser origins. Empty = CORS disabled (same-origin deployment through the reverse proxy / Next.js rewrites).
`APP_TIMEZONE`	backend	no	no	`Asia/Colombo`	`Asia/Colombo`	IANA time zone of the dispensary. Defines 'today' for queue sessions, session-date validation and age calculation. Instants are always stored in UTC.
`APP_BOOTSTRAP_ADMIN_USERNAME`	backend	no	no	-	-	If set and the users table is empty at startup, an ADMIN with this username is created (must_change_password = true).

Variable	Component	Required	Secret	Default	Local dev value	Description
APP_BOOTSTRAP_ADMIN_PASSWORD	backend	no	yes	-	-	Initial password for the bootstrap admin (min 12 chars). Remove from the environment after first login.
APP_SWAGGER_ENABLED	backend	no	no	false	true	Exposes /swagger-ui.html and /v3/api-docs. Keep false in production unless protected by the network.
APP_RATE_LIMIT_LOGIN_PER_MINUTE	backend	no	no	10	10	Max POST /auth/login requests per client IP per minute.
APP_RATE_LIMIT_PUBLIC_PER_MINUTE	backend	no	no	120	600	Max /api/v1/public/** requests per client IP per minute.
APP_RETENTION_QUEUE_PII_DAYS	backend	no	no	90	90	Queue entries older than this have name, phone and notes removed by the nightly retention job.
SERVER_FORWARD_HEADERS_STRATEGY	backend	no	no	none	native	Set to native only when the API is reachable exclusively through a trusted reverse proxy, so X-Forwarded-For gives the real client IP for audit and rate limiting.
LOGGING_STRUCTURED_FORMAT_CONSOLE	backend	no	no	ecs	-	Spring Boot structured logging format for console output (ecs recommended in production; empty = human-readable).
API_PROXY_TARGET	frontend	yes	no	-	http://localhost:8080	Server-side only (NOT NEXT_PUBLIC). Origin the Next.js rewrite forwards /api/* to, e.g. http://backend:8080 inside Docker Compose.
NEXT_PUBLIC_API_BASE_PATH	frontend	no	no	/api/v1	/api/v1	Path prefix used by the browser API client. Relative so the browser always calls the same origin.
NEXT_PUBLIC_APP_NAME	frontend	no	no	Dispensary Management System	DMS (dev)	Name shown in the header and page titles.
NEXT_PUBLIC_QUEUE_POLL_INTERVAL_MS	frontend	no	no	5000	5000	Polling interval for public and staff queue views (min 2000).
NEXT_PUBLIC_PRESCRIPTION_POLL_INTERVAL_MS	frontend	no	no	10000	10000	Polling interval for the pharmacist prescription queue and doctor status views (min 5000).
NEXT_PUBLIC_IDLE_TIMEOUT_MINUTES	frontend	no	no	30	30	Staff session is logged out after this many minutes without keyboard/mouse/touch activity.
POSTGRES_DB	database	yes	no	-	dms	Database created by the official postgres image.
POSTGRES_USER	database	yes	no	-	dms	Owner user created by the postgres image.
POSTGRES_PASSWORD	database	yes	yes	-	dms_dev_password	Password of POSTGRES_USER.

3. Data model

3.1 Identifier strategy

- **Primary keys are UUID v4** (`UUID` columns). Hibernate generates them (`@GeneratedValue(strategy = GenerationType.UUID)`); the database default `gen_random_uuid()` serves seed scripts. UUIDs are not guessable or countable in URLs, need no sequence round-trip and are safe for future data merges. The storage overhead (16 bytes) is irrelevant at this scale.
- **Human identifiers** are separate columns: `patients.patient_number` (staff-entered card number, unique), `prescriptions.prescription_number` (sequence, shown as `RX-000123`) and `queue_entries.queue_number` (1-999 per session, gap-free).
- Composite natural keys are used only where they express an invariant: `user_roles(user_id, role)`.

3.2 Table inventory - why each table exists

Table	Why it exists	Rows (A-05, 5 years)
`users`	Staff accounts and login state (lockout, password change)	< 100
`user_roles`	A user can hold several roles (doctor + pharmacist); roles are a fixed enum, so no `roles` table	< 200
`refresh_tokens`	Server-side revocation and rotation of refresh tokens	~ 50 000 (purged)
`patients`	Demographic master record and business identifier	~ 50 000
`consultations`	The medical history: one immutable row per encounter	~ 365 000
`consultation_diagnoses`	1-10 conditions per consultation; source of "previous medical conditions"	~ 600 000
`medicines`	Consistent names and the active/unavailable flag for prescribing	< 2 000
`prescriptions`	Prescription header and lifecycle status	~ 300 000
`prescription_items`	What the doctor prescribed (immutable)	~ 1 000 000
`dispensings`	Who dispensed, when, overall outcome - one per prescription	~ 290 000
`dispensing_items`	What was actually handed over per item	~ 950 000
`queue_sessions`	A numbered queue per clinic/date with the counters	~ 4 000
`queue_entries`	Bookings / queue positions (one table: each booking is exactly one entry)	~ 550 000
`idempotency_keys`	Exactly-once create operations under retries	< 10 000 (24 h)
`audit_events`	Append-only audit trail	~ 5 000 000

Deliberately not created: a `roles` table (fixed enum, no attributes); `doctors` / `staff_profiles` (the only profile attribute, registration number, is a nullable column on `users`); a separate `bookings` table (a booking is a queue entry 1:1); a diagnosis catalogue (free text + optional ICD-10 code in release 1.0); a dispensing-status column on prescription items (what was dispensed lives in `dispensing_items` so prescribed data stays immutable); inventory tables (out of scope).

3.3 Entity relationship diagrams

The diagrams are generated from the database catalogue after applying the migration. In this PDF they are rendered with Graphviz for legibility; the equivalent Mermaid sources (`diagrams/database-erd.mmd` and `diagrams/erd-*.mmd`) are generated from the same model. Notation: `*` = NOT NULL; crow's foot = many; bar = exactly one; circle = optional.

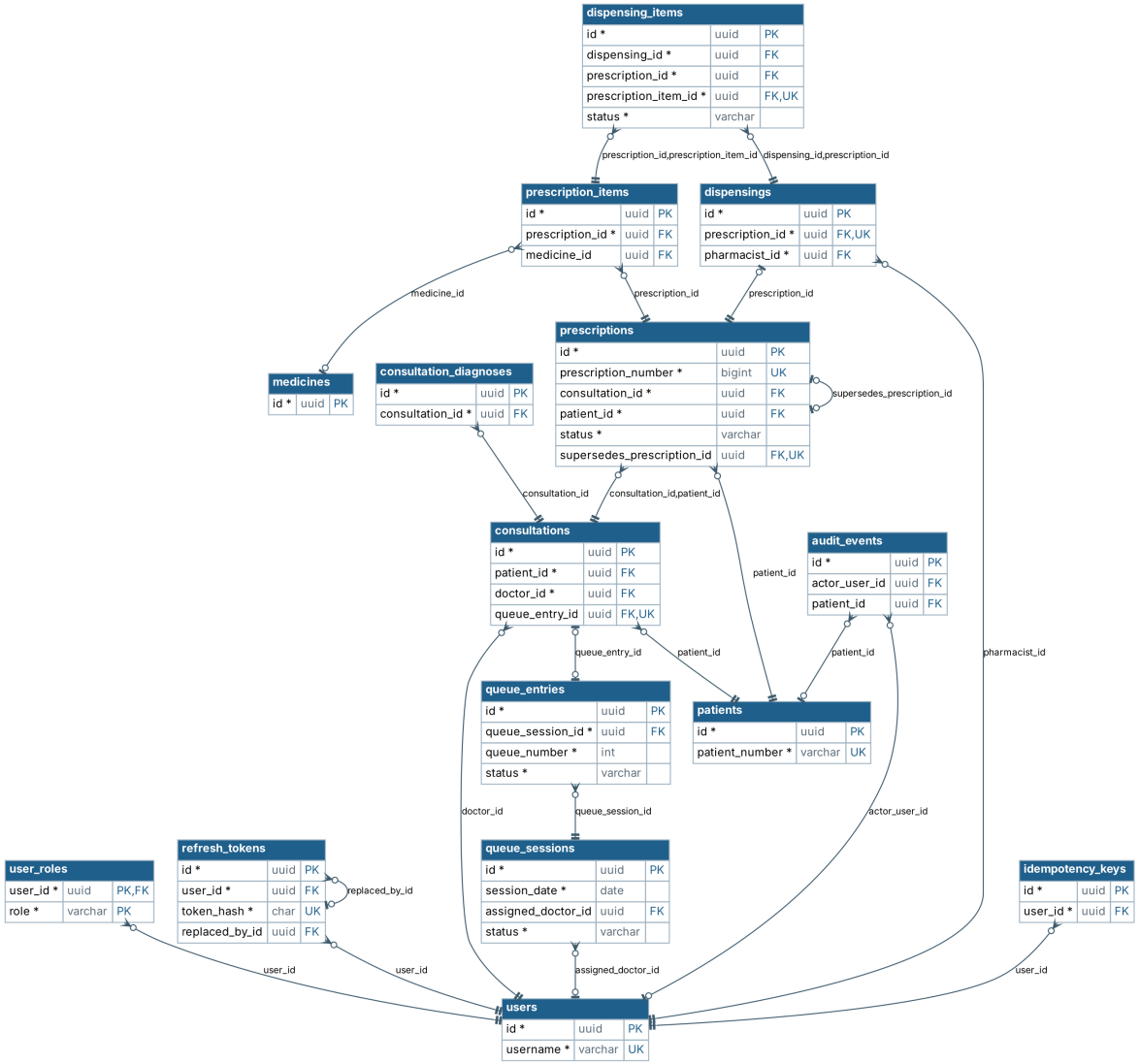


Figure 13. Database ERD - overview with keys and status columns (attribution foreign keys to users omitted) (source: diagrams/database-erd.mmd)

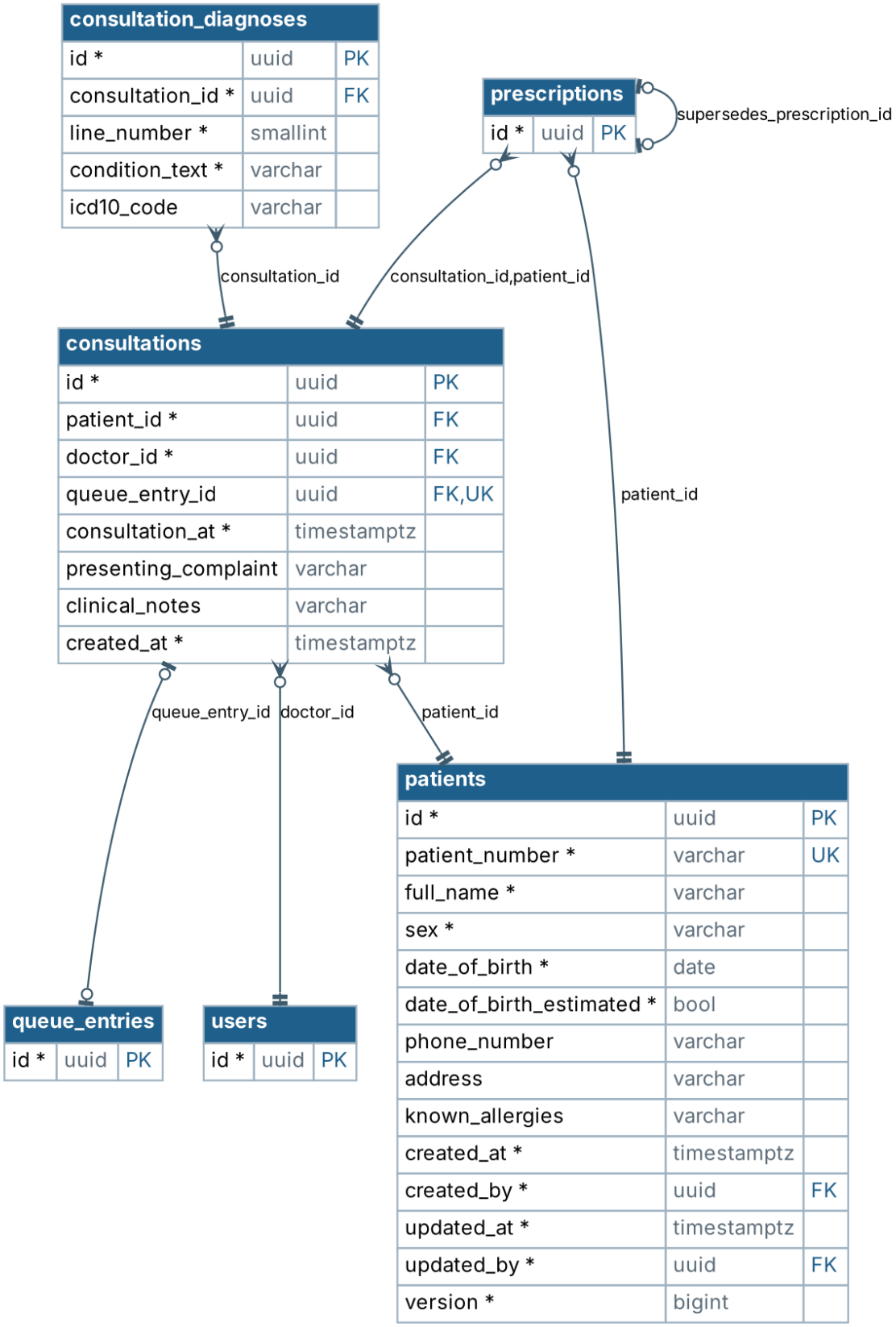


Figure 14. ERD - patients, consultations and diagnoses (all columns) (source: diagrams/erd-patient-consultation.mmd)

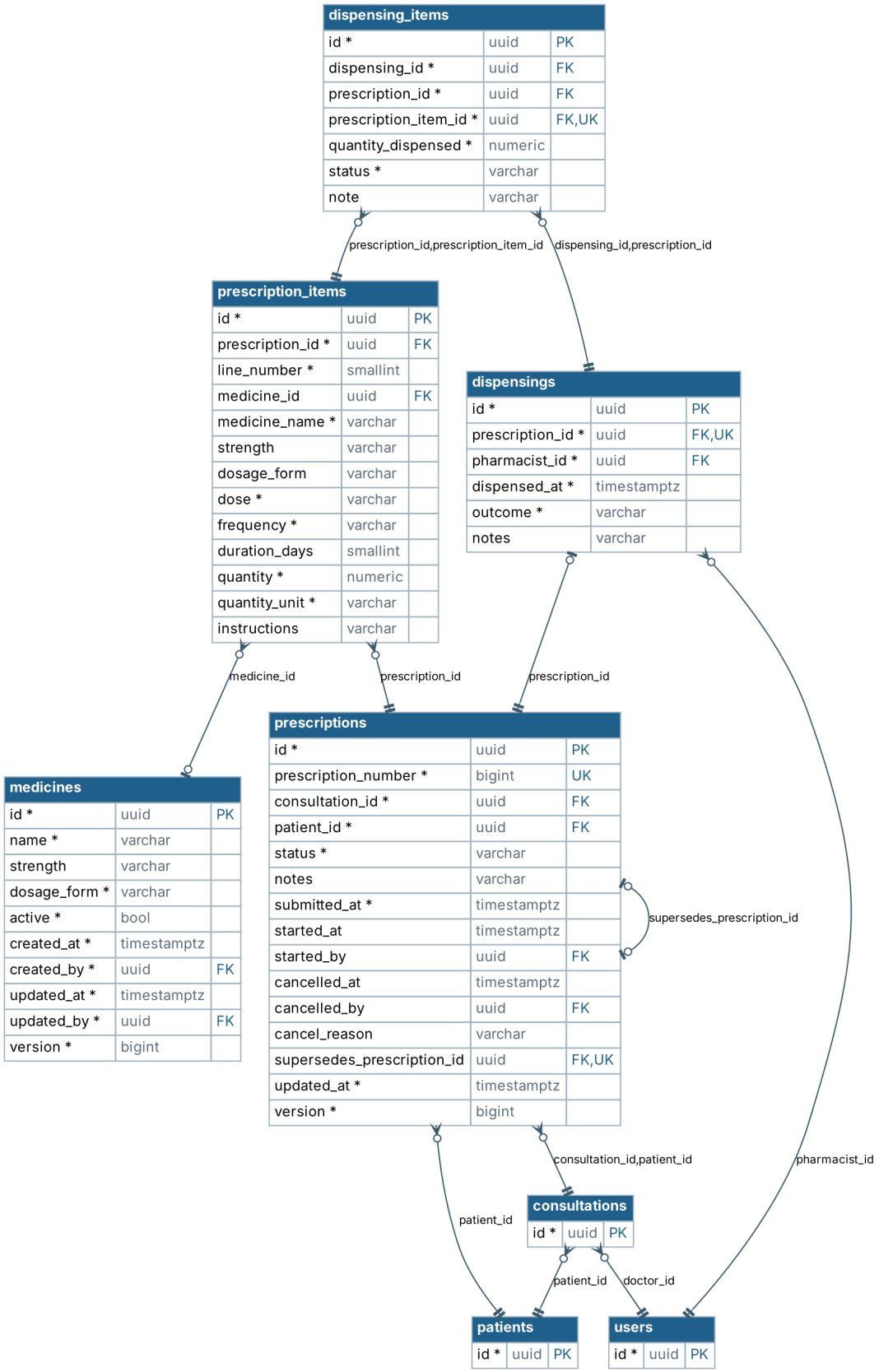


Figure 15. ERD - prescriptions, items, medicines and dispensing (all columns) (source: diagrams/erd-prescription-dispensing.mmd)

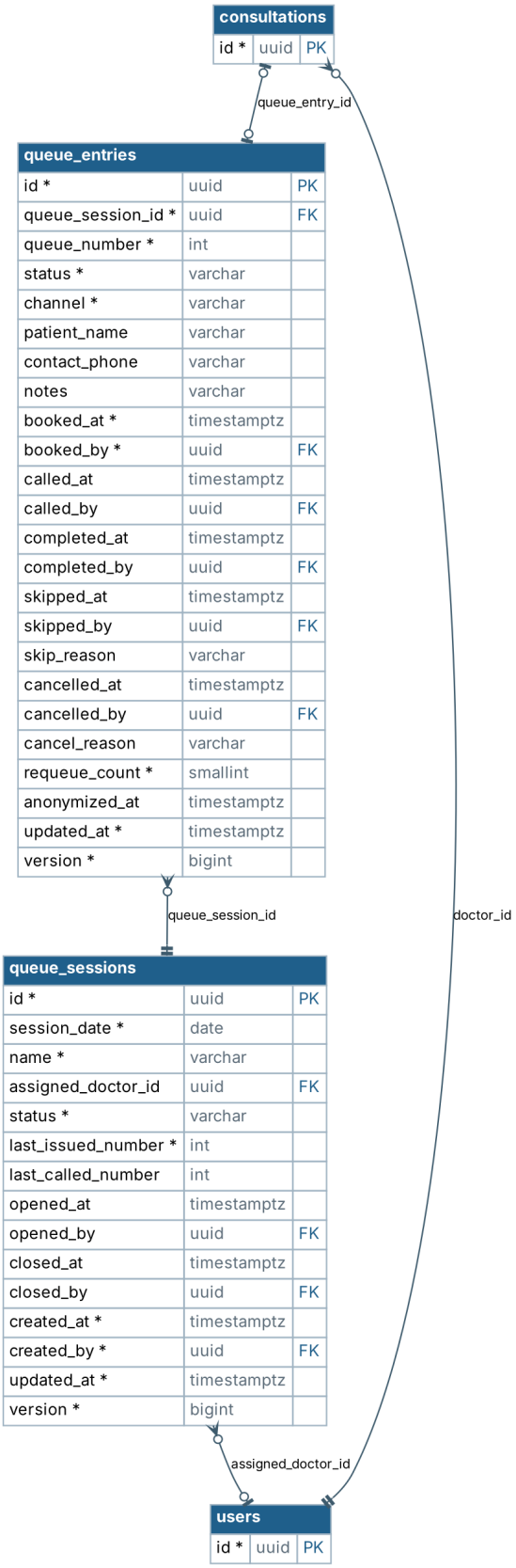


Figure 16. ERD - queue sessions and queue entries (all columns) (source: diagrams/erd-queue.mmd)

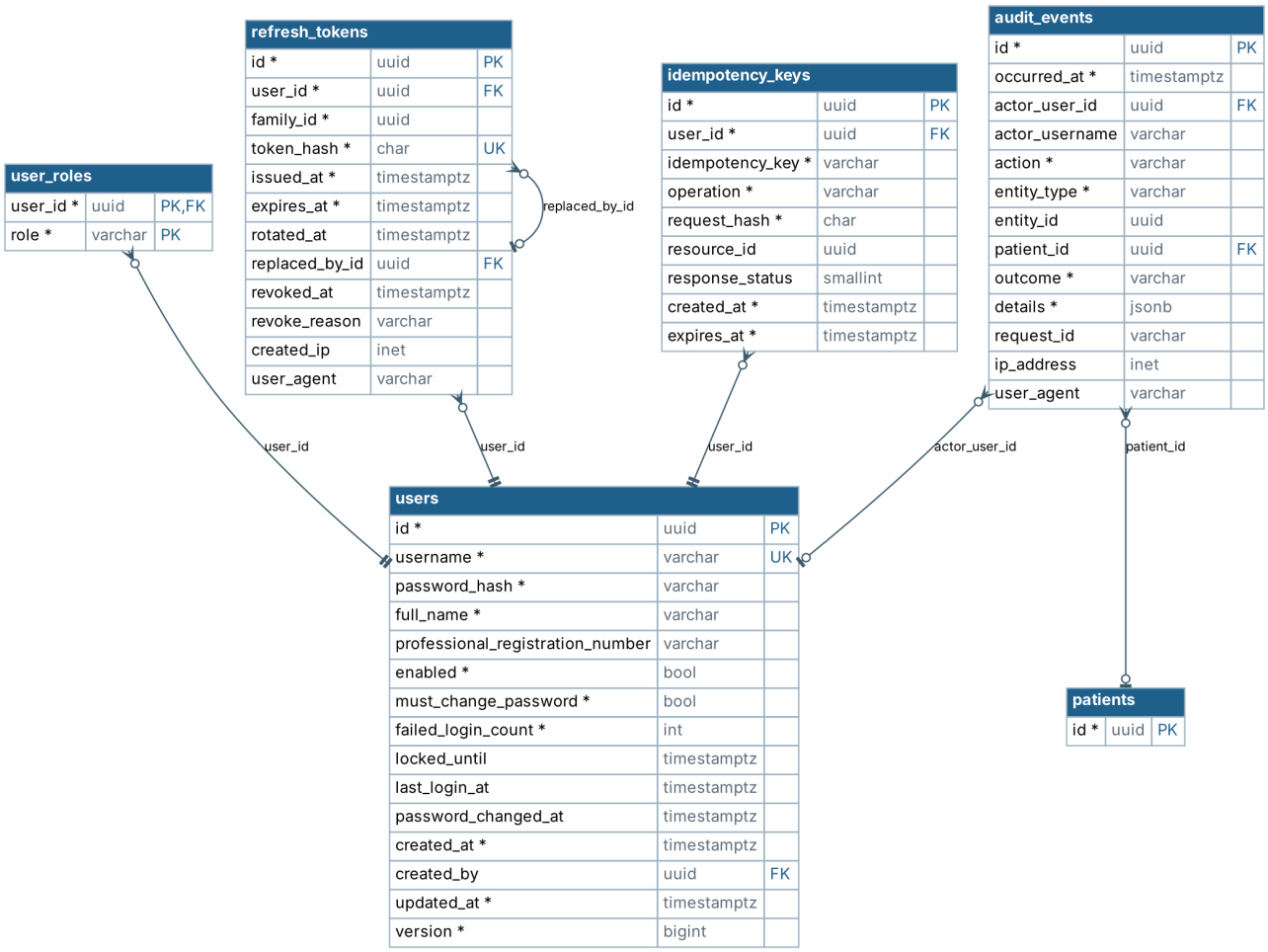


Figure 17. ERD - users, roles, refresh tokens, idempotency keys and audit events (all columns) (source: diagrams/erd-security-audit.mmd)

3.4 Primary and foreign key design

- Every table except `user_roles` has a single-column UUID primary key `id`.
- All foreign keys use `ON DELETE RESTRICT` (the default) because nothing clinical is deleted (BR-G-02); only `user_roles` cascades from `users`.
- **Composite foreign keys enforce cross-row consistency:** `prescriptions(consultation_id, patient_id)` references `consultations(id, patient_id)`, so a prescription's patient always equals its consultation's patient; `dispensing_items(dispensing_id, prescription_id)` and `dispensing_items(prescription_item_id, prescription_id)` guarantee a dispensed item belongs to the dispensed prescription.
- Partial unique index `uq_prescriptions_active_per_consultation` (non-cancelled prescriptions) enforces "at most one active prescription".
- `UNIQUE(prescription_id)` on `dispensings` and `UNIQUE(prescription_item_id)` on `dispensing_items` make duplicate dispensing impossible.
- `UNIQUE(queue_session_id, queue_number)` backs the gap-free numbering; `UNIQUE(queue_entry_id)` on consultations prevents double linking.

3.5 Normalisation decisions

The schema is in third normal form with four deliberate, controlled exceptions:

Exception	Why	How consistency is guaranteed
`prescriptions.patient_id` (derivable via consultation)	Pharmacist queue and history queries join patients directly	Composite FK to `consultations(id, patient_id)`
`prescription_items.medicine_name/strength/dosage_form` copied from `medicines`	Historical record must not change when the catalogue is edited (legal record of what was prescribed)	Copied once at creation; items are immutable
`dispensing_items.prescription_id`	Needed only to express the two composite FKs	Composite FKs
`queue_sessions.last_issued_number/last_called_number` (derivable from entries)	Gap-free numbering and compare-and-set require a single lockable counter row	Updated in the same transaction as the entry; CHECK `last_called <= last_issued`

Counts shown in the UI (waiting, serving, ...) are always computed from `queue_entries`, never stored.

3.6 Date of birth and age

`patients.date_of_birth` is `NOT NULL`. When the real date is unknown, staff enter an estimated age N and the service stores `today(APP_TIMEZONE) - N years` with `date_of_birth_estimated = TRUE` (SRS BR-PAT-02). Age is computed on read (`Period.between(dob, today).getYears()`), so it is always current and never stored. The UI shows estimated ages as "~70 y".

3.7 Medical history representation

There is no "medical history" column. The history is the set of `consultations` of a patient (insert-only) with their `consultation_diagnoses`, `prescriptions` (including cancelled and superseded ones), `prescription_items`, `dispensings` and `dispensing_items`. "Previous medical conditions" is an aggregation of `consultation_diagnoses` (example query Q5). Nothing is overwritten when a new consultation is created.

3.8 Indexing strategy

Access path	Index	Used by
Patient by number	`uq_patients_patient_number` (unique btree)	getPatientByNumber
Patient name search (`ILIKE %x%`)	`ix_patients_full_name_trgm` (GIN, pg_trgm)	searchPatients
Patient by phone	`ix_patients_phone_number` (partial)	searchPatients
History page newest first	`ix_consultations_patient_at (patient_id, consultation_at DESC)`	getMedicalHistory
Doctor's own consultations / prescriptions	`ix_consultations_doctor_at`	listPrescriptions (doctor)
Pharmacist queue	`ix_prescriptions_status_submitted (status, submitted_at)`	listPrescriptions
Prescriptions of a consultation / patient	`ix_prescriptions_consultation`, `ix_prescriptions_patient_submitted`	history batch load
Items / dispensing of prescriptions	`uq_prescription_items_line`, `uq_dispensings_prescription`, `ix_dispensing_items_dispensing`	batch loads
Next waiting entry, counts by status	`ix_queue_entries_session_status_number`	callNextPatient, getQueueSession, public

Access path	Index	Used by
Duplicate-phone check	`ix_queue_entries_active_phone` (partial: WAITING/SERVING)	createBooking
Sessions of a day	`ix_queue_sessions_date_status`, `uq_queue_sessions_date_name`	listQueueSessions, public list
Medicine autocomplete	`ix_medicines_name_trgm`	listMedicines
Audit by patient / actor / time / entity	`ix_audit_events_patient`, `ix_audit_events_actor`, `ix_audit_events_occurred_at`, `ix_audit_events_entity`	listAuditEvents
Housekeeping	`ix_refresh_tokens_expires_at`, `ix_idempotency_keys_expires_at`, `ix_queue_entries_booked_at` (partial)	retention job

Every foreign key used in joins is covered by an index (directly or as the leading column of a unique constraint). Indexes are not added for attribution columns (`*_by`) because they are never filtered on.

4. Data dictionary

Generated from the database catalogue after applying `V1__initial_schema.sql`. Column descriptions are the `COMMENT ON` texts of the migration.

4.1 users

Purpose. Staff accounts that can authenticate. Users are never hard-deleted; they are disabled (enabled = false) so that historical references stay valid.

Column	Type	Null	Default	Keys	Description
`id`	uuid	NOT NULL	gen_random_uuid()	PK	Surrogate key (UUID).
`username`	character varying(50)	NOT NULL	-	UK	Login name, stored lower-case. Pattern ^[a-z0-9._-]{3,50}$.
`password_hash`	character varying(255)	NOT NULL	-	-	Spring Security DelegatingPasswordEncoder value, e.g. {bcrypt}$2b$12$... Never returned by the API.
`full_name`	character varying(150)	NOT NULL	-	-	Display name shown on prescriptions and audit views.
`professional_registration_number`	character varying(50)	null	-	-	Optional professional registration number (e.g. medical council or pharmacy council number).
`enabled`	boolean	NOT NULL	true	-	FALSE = account disabled; login and token refresh are refused.
`must_change_password`	boolean	NOT NULL	true	-	TRUE after account creation or admin password reset; all endpoints except auth/me, auth/change-password and auth/logout return 403 PASSWORD_CHANGE_REQUIRED.
`failed_login_count`	integer	NOT NULL	0	-	Consecutive failed logins; reset to 0 on success.
`locked_until`	timestamp with time zone	null	-	-	Login refused until this instant (set after 5 consecutive failures, 15 minutes).
`last_login_at`	timestamp with time zone	null	-	-	Instant of the last successful login.
`password_changed_at`	timestamp with time zone	null	-	-	Instant the password was last set by the user.
`created_at`	timestamp with time zone	NOT NULL	now()	-	Creation instant (UTC).
`created_by`	uuid	null	-	FK → users	Admin who created the account; NULL for the bootstrap admin.

Column	Type	Null	Default	Keys	Description
`updated_at`	timestamp with time zone	NOT NULL	now()	-	Last modification instant (UTC).
`version`	bigint	NOT NULL	0	-	Optimistic-locking version (JPA @Version).

Constraints and indexes

Name	Kind	Definition
`ck_users_failed_login_count`	CHECK	`CHECK ((failed_login_count >= 0))`
`ck_users_full_name_not_blank`	CHECK	`CHECK ((btrim((full_name)::text) <> ''::text))`
`ck_users_username_format`	CHECK	`CHECK (((username)::text ~ '^[a-z0-9._-]{3,50}$'::text))`
`fk_users_created_by`	FOREIGN KEY	`FOREIGN KEY (created_by) REFERENCES users(id)`
`pk_users`	PRIMARY KEY	`PRIMARY KEY (id)`
`uq_users_username`	UNIQUE	`UNIQUE (username)`

Referenced by: `audit_events`, `consultations`, `dispensings`, `idempotency_keys`, `medicines`, `patients`, `prescriptions`, `queue_entries`, `queue_sessions`, `refresh_tokens`, `user_roles`.

4.2 user_roles

Purpose. Role grants. Roles are a fixed code-level enumeration, so no separate roles table is needed.

Column	Type	Null	Default	Keys	Description
`user_id`	uuid	NOT NULL	-	FK → users PK (composite)	Granted user.
`role`	character varying(20)	NOT NULL	-	PK (composite)	ADMIN | DOCTOR | PHARMACIST.

Constraints and indexes

Name	Kind	Definition
`ck_user_roles_role`	CHECK	`CHECK (((role)::text = ANY ((ARRAY['ADMIN'::character varying, 'DOCTOR'::character varying, 'PHARMACIST'::character varying])::text[])))`
`fk_user_roles_user`	FOREIGN KEY	`FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE`

Name	Kind	Definition
`pk_user_roles`	PRIMARY KEY	`PRIMARY KEY (user_id, role)`
`ix_user_roles_role`	INDEX	`btree (role)`

4.3 refresh_tokens

Purpose. Opaque refresh tokens (only the SHA-256 hash is stored). Rotated on every use; reuse of a rotated token outside the grace window revokes the whole family.

Column	Type	Null	Default	Keys	Description
`id`	uuid	NOT NULL	gen_random_uuid()	PK	
`user_id`	uuid	NOT NULL	-	FK → users	
`family_id`	uuid	NOT NULL	-	-	All tokens descending from one login share a family; the family expires APP_JWT_REFRESH_TOKEN_TTL after login.
`token_hash`	character(64)	NOT NULL	-	UK	Lower-case hex SHA-256 of the raw token. The raw token exists only in the HttpOnly cookie.
`issued_at`	timestamp with time zone	NOT NULL	now()	-	Issue instant.
`expires_at`	timestamp with time zone	NOT NULL	-	-	Absolute expiry; equals the family expiry (login instant + refresh TTL).
`rotated_at`	timestamp with time zone	null	-	-	Set when the token was exchanged for a successor.
`replaced_by_id`	uuid	null	-	FK → refresh_tokens	Successor token after rotation.
`revoked_at`	timestamp with time zone	null	-	-	Set when revoked; revoked tokens are never accepted.
`revoke_reason`	character varying(30)	null	-	-	LOGOUT | REUSE_DETECTED | PASSWORD_CHANGED | PASSWORD_RESET | USER_DISABLED.
`created_ip`	inet	null	-	-	Client IP at issue (for security investigation).
`user_agent`	character varying(255)	null	-	-	Truncated User-Agent at issue.

Constraints and indexes

Name	Kind	Definition
ck_refresh_tokens_expiry	CHECK	CHECK ((expires_at > issued_at))
ck_refresh_tokens_hash_hex	CHECK	CHECK ((token_hash ~ '^[0-9a-f]{64}$'::text))
ck_refresh_tokens_revoke_reason	CHECK	CHECK (((revoke_reason IS NULL) OR ((revoke_reason)::text = ANY ((ARRAY['LOGOUT'::character varying, 'REUSE_DETECTED'::character varying, 'PASSWORD_CHANGED'::character varying, 'PASSWORD_RESET'::character varying, 'USER_DISABLED'::character varying])::text[]))))
ck_refresh_tokens_revoked_consistency	CHECK	CHECK (((revoked_at IS NULL) = (revoke_reason IS NULL)))
fk_refresh_tokens_replaced_by	FOREIGN KEY	FOREIGN KEY (replaced_by_id) REFERENCES refresh_tokens(id)
fk_refresh_tokens_user	FOREIGN KEY	FOREIGN KEY (user_id) REFERENCES users(id)
pk_refresh_tokens	PRIMARY KEY	PRIMARY KEY (id)
uq_refresh_tokens_token_hash	UNIQUE	UNIQUE (token_hash)
ix_refresh_tokens_expires_at	INDEX	btree (expires_at)
ix_refresh_tokens_family	INDEX	btree (family_id)
ix_refresh_tokens_user	INDEX	btree (user_id)

4.4 patients

Purpose. One row per patient. Clinical history is NOT stored here; it is the set of consultations that reference the patient.

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK	
patient_number	character varying(20)	NOT NULL	-	UK	Business identifier printed on the clinic card; entered by staff, normalised to upper case, immutable after registration.
full_name	character varying(150)	NOT NULL	-	-	Patient full name.
sex	character varying(10)	NOT NULL	-	-	MALE | FEMALE | OTHER | UNKNOWN.
date_of_birth	date	NOT NULL	-	-	Exact DOB, or a DOB derived from an estimated age (see date_of_birth_estimated). Age is always computed, never stored.

Column	Type	Null	Default	Keys	Description
date_of_birth_estimated	boolean	NOT NULL	false	-	TRUE when the DOB was derived from an estimated age because the real DOB is unknown.
phone_number	character varying(16)	null	-	-	Optional contact number, digits with optional leading +, 7-15 digits.
address	character varying(300)	null	-	-	Optional postal address.
known_allergies	character varying(1000)	null	-	-	Clinician-entered allergy statement shown to doctors and pharmacists. Changes are audited.
created_at	timestamp with time zone	NOT NULL	now()	-	
created_by	uuid	NOT NULL	-	FK → users	Doctor who registered the patient.
updated_at	timestamp with time zone	NOT NULL	now()	-	
updated_by	uuid	NOT NULL	-	FK → users	Last user who changed demographics.
version	bigint	NOT NULL	0	-	Optimistic-locking version; PUT requests must send the current value.

Constraints and indexes

Name	Kind	Definition
ck_patients_dob_min	CHECK	CHECK ((date_of_birth >= '1900-01-01'::date))
ck_patients_full_name_not_blank	CHECK	CHECK ((btrim((full_name)::text) <> ''::text))
ck_patients_patient_number_format	CHECK	CHECK (((patient_number)::text ~ '^[A-Z0-9][A-Z0-9-]{2,19}$'::text))
ck_patients_phone_format	CHECK	CHECK (((phone_number IS NULL) OR ((phone_number)::text ~ '^\+?[0-9]{7,15}$'::text)))
ck_patients_sex	CHECK	CHECK (((sex)::text = ANY ((ARRAY['MALE'::character varying, 'FEMALE'::character varying, 'OTHER'::character varying, 'UNKNOWN'::character varying])::text[])))
fk_patients_created_by	FOREIGN KEY	FOREIGN KEY (created_by) REFERENCES users(id)
fk_patients_updated_by	FOREIGN KEY	FOREIGN KEY (updated_by) REFERENCES users(id)

Name	Kind	Definition
pk_patients	PRIMARY KEY	PRIMARY KEY (id)
uq_patients_patient_number	UNIQUE	UNIQUE (patient_number)
ix_patients_full_name_trgm	INDEX	gin (full_name gin_trgm_ops)
ix_patients_phone_number	INDEX	btree (phone_number) WHERE (phone_number IS NOT NULL)

Referenced by: audit_events, consultations, prescriptions.

4.5 queue_sessions

Purpose. A queue with its own number sequence. Numbering restarts at 1 for every session; sessions are per date, so numbering restarts at least daily.

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK	
session_date	date	NOT NULL	-	-	Business date of the session in APP_TIMEZONE.
name	character varying(100)	NOT NULL	-	-	Display label, e.g. "Morning OPD - Room 1". Unique per date (case-insensitive).
assigned_doctor_id	uuid	null	-	FK → users	Optional doctor who normally serves this queue (informational; any doctor may call).
status	character varying(12)	NOT NULL	'SCHEDULED'	-	SCHEDULED (bookings allowed, no calls) | OPEN (bookings and calls) | CLOSED (terminal).
last_issued_number	integer	NOT NULL	0	-	Counter of issued queue numbers. Incremented under a row lock in the same transaction that inserts the queue entry, so numbers are unique and gap-free.
last_called_number	integer	null	-	-	Number most recently called (the public "now serving" number). Used as the compare-and-set token for call-next.
opened_at	timestamp with time zone	null	-	-	Instant the session was opened.
opened_by	uuid	null	-	FK → users	
closed_at	timestamp with time zone	null	-	-	Instant the session was closed.

Column	Type	Null	Default	Keys	Description
closed_by	uuid	null	-	FK → users	
created_at	timestamp with time zone	NOT NULL	now()	-	
created_by	uuid	NOT NULL	-	FK → users	
updated_at	timestamp with time zone	NOT NULL	now()	-	
version	bigint	NOT NULL	0	-	Incremented on every queue mutation (booking, call, complete, skip, cancel, requeue); exposed as the public ETag.

Constraints and indexes

Name	Kind	Definition
ck_queue_sessions_last_called	CHECK	CHECK (((last_called_number IS NULL) OR ((last_called_number >= 1) AND (last_called_number <= last_issued_number))))
ck_queue_sessions_last_issued	CHECK	CHECK (((last_issued_number >= 0) AND (last_issued_number <= 999)))
ck_queue_sessions_name_not_blank	CHECK	CHECK ((btrim((name)::text) <> ''::text))
ck_queue_sessions_status	CHECK	CHECK (((status)::text = ANY ((ARRAY['SCHEDULED'::character varying, 'OPEN'::character varying, 'CLOSED'::character varying])::text[])))
ck_queue_sessions_status_timestamps	CHECK	CHECK (((((status)::text = 'SCHEDULED'::text) AND (opened_at IS NULL) AND (closed_at IS NULL)) OR (((status)::text = 'OPEN'::text) AND (opened_at IS NOT NULL) AND (closed_at IS NULL)) OR (((status)::text = 'CLOSED'::text) AND (closed_at IS NOT NULL))))
fk_queue_sessions_closed_by	FOREIGN KEY	FOREIGN KEY (closed_by) REFERENCES users(id)
fk_queue_sessions_created_by	FOREIGN KEY	FOREIGN KEY (created_by) REFERENCES users(id)
fk_queue_sessions_doctor	FOREIGN KEY	FOREIGN KEY (assigned_doctor_id) REFERENCES users(id)
fk_queue_sessions_opened_by	FOREIGN KEY	FOREIGN KEY (opened_by) REFERENCES users(id)
pk_queue_sessions	PRIMARY KEY	PRIMARY KEY (id)

Name	Kind	Definition
ix_queue_sessions_date_status	INDEX	btree (session_date, status)
uq_queue_sessions_date_name	UNIQUE INDEX	btree (session_date, lower((name)::text))

Referenced by: queue_entries.

4.6 queue_entries

Purpose. A booking. Creating a booking (telephone or walk-in) creates exactly one queue entry, so bookings and queue entries share one table.

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK	
queue_session_id	uuid	NOT NULL	-	FK → queue_sessions UK (composite)	Session the entry belongs to.
queue_number	integer	NOT NULL	-	UK (composite)	Number given to the patient; unique and gap-free within the session, never reused.
status	character varying(12)	NOT NULL	'WAITING'	-	WAITING | SERVING | COMPLETED | SKIPPED | CANCELLED (queue lifecycle, distinct from prescription statuses).
channel	character varying(10)	NOT NULL	-	-	PHONE (telephone booking) | WALK_IN.
patient_name	character varying(150)	null	-	-	Name given by the caller. Staff-only; never exposed publicly. Set to NULL by the retention job (anonymized_at).
contact_phone	character varying(16)	null	-	-	Optional callback number. Staff-only; used for duplicate-booking detection. Cleared by the retention job.
notes	character varying(300)	null	-	-	Short non-clinical booking note (e.g. "elderly, needs wheelchair"). Cleared by the retention job.
booked_at	timestamp with time zone	NOT NULL	now()	-	
booked_by	uuid	NOT NULL	-	FK → users	Staff member who registered the booking.
called_at	timestamp with time zone	null	-	-	Last time the entry was called (moved to SERVING).
called_by	uuid	null	-	FK → users	

Column	Type	Null	Default	Keys	Description
completed_at	timestamp with time zone	null	-	-	
completed_by	uuid	null	-	FK → users	
skipped_at	timestamp with time zone	null	-	-	
skipped_by	uuid	null	-	FK → users	
skip_reason	character varying(300)	null	-	-	
cancelled_at	timestamp with time zone	null	-	-	
cancelled_by	uuid	null	-	FK → users	
cancel_reason	character varying(500)	null	-	-	
requeue_count	smallint	NOT NULL	0	-	Number of times a skipped entry was returned to WAITING.
anonymized_at	timestamp with time zone	null	-	-	Set when the retention job removed name, phone and notes.
updated_at	timestamp with time zone	NOT NULL	now()	-	
version	bigint	NOT NULL	0	-	Optimistic-locking version.

Constraints and indexes

Name	Kind	Definition
ck_queue_entries_channel	CHECK	CHECK (((channel)::text = ANY ((ARRAY['PHONE'::character varying, 'WALK_IN'::character varying])::text[])))
ck_queue_entries_name_present	CHECK	CHECK (((patient_name IS NOT NULL) OR (anonymized_at IS NOT NULL)))
ck_queue_entries_number	CHECK	CHECK (((queue_number >= 1) AND (queue_number <= 999)))
ck_queue_entries_phone_format	CHECK	CHECK (((contact_phone IS NULL) OR ((contact_phone)::text ~ '^\+?[0-9]{7,15}$'::text)))
ck_queue_entries_requeue_count	CHECK	CHECK ((requeue_count >= 0))
ck_queue_entries_status	CHECK	CHECK (((status)::text = ANY ((ARRAY['WAITING'::character varying, 'SERVING'::character varying, 'COMPLETED'::character varying, 'SKIPPED'::character varying, 'CANCELLED'::character varying])::text[])))

Name	Kind	Definition
ck_queue_entries_status_timestamps	CHECK	CHECK (((((status)::text <> 'SERVING'::text) OR (called_at IS NOT NULL)) AND (((status)::text <> 'COMPLETED'::text) OR (completed_at IS NOT NULL)) AND (((status)::text <> 'SKIPPED'::text) OR (skipped_at IS NOT NULL)) AND (((status)::text <> 'CANCELLED'::text) OR ((cancelled_at IS NOT NULL) AND (cancel_reason IS NOT NULL)))))
fk_queue_entries_booked_by	FOREIGN KEY	FOREIGN KEY (booked_by) REFERENCES users(id)
fk_queue_entries_called_by	FOREIGN KEY	FOREIGN KEY (called_by) REFERENCES users(id)
fk_queue_entries_cancelled_by	FOREIGN KEY	FOREIGN KEY (cancelled_by) REFERENCES users(id)
fk_queue_entries_completed_by	FOREIGN KEY	FOREIGN KEY (completed_by) REFERENCES users(id)
fk_queue_entries_session	FOREIGN KEY	FOREIGN KEY (queue_session_id) REFERENCES queue_sessions(id)
fk_queue_entries_skipped_by	FOREIGN KEY	FOREIGN KEY (skipped_by) REFERENCES users(id)
pk_queue_entries	PRIMARY KEY	PRIMARY KEY (id)
uq_queue_entries_session_number	UNIQUE	UNIQUE (queue_session_id, queue_number)
ix_queue_entries_active_phone	INDEX	btree (queue_session_id, contact_phone) WHERE (((status)::text = ANY ((ARRAY['WAITING'::character varying, 'SERVING'::character varying])::text[])) AND (contact_phone IS NOT NULL))
ix_queue_entries_booked_at	INDEX	btree (booked_at) WHERE (anonymized_at IS NULL)
ix_queue_entries_session_status_number	INDEX	btree (queue_session_id, status, queue_number)

Referenced by: `consultations`.

4.7 consultations

Purpose. Medical history is the ordered set of consultations of a patient. Rows are insert-only: no UPDATE or DELETE paths exist in v1.

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK UK (composite)	

Column	Type	Null	Default	Keys	Description
patient_id	uuid	NOT NULL	-	FK → patients UK (composite)	Patient seen.
doctor_id	uuid	NOT NULL	-	FK → users	Doctor who recorded the consultation (the authenticated user).
queue_entry_id	uuid	null	-	FK → queue_entries UK	Optional link to the queue entry that was being served; at most one consultation per entry.
consultation_at	timestamp with time zone	NOT NULL	now()	-	Instant the consultation was submitted (server clock).
presenting_complaint	character varying(1000)	null	-	-	Optional presenting complaint / reason for visit.
clinical_notes	character varying(5000)	null	-	-	Optional clinical notes. Sensitive: never logged, never shown to pharmacists.
created_at	timestamp with time zone	NOT NULL	now()	-	Row creation instant.

Constraints and indexes

Name	Kind	Definition
fk_consultations_doctor	FOREIGN KEY	FOREIGN KEY (doctor_id) REFERENCES users(id)
fk_consultations_patient	FOREIGN KEY	FOREIGN KEY (patient_id) REFERENCES patients(id)
fk_consultations_queue_entry	FOREIGN KEY	FOREIGN KEY (queue_entry_id) REFERENCES queue_entries(id)
pk_consultations	PRIMARY KEY	PRIMARY KEY (id)
uq_consultations_id_patient	UNIQUE	UNIQUE (id, patient_id)
uq_consultations_queue_entry	UNIQUE	UNIQUE (queue_entry_id)
ix_consultations_doctor_at	INDEX	btree (doctor_id, consultation_at DESC)
ix_consultations_patient_at	INDEX	btree (patient_id, consultation_at DESC)

Referenced by: consultation_diagnoses, prescriptions.

4.8 consultation_diagnoses

Purpose. One row per condition/diagnosis recorded in a consultation (1-10 per consultation). "Previous medical conditions" are derived from these rows.

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK	
consultation_id	uuid	NOT NULL	-	FK → consultations UK (composite)	

Column	Type	Null	Default	Keys	Description
line_number	smallint	NOT NULL	-	UK (composite)	Order within the consultation, 1 = primary.
condition_text	character varying(500)	NOT NULL	-	-	Clinician-entered condition or diagnosis text.
icd10_code	character varying(10)	null	-	-	Optional ICD-10 code entered by the doctor (format-checked only; no code catalogue in v1).

Constraints and indexes

Name	Kind	Definition
ck_consultation_diagnoses_icd10	CHECK	CHECK (((icd10_code IS NULL) OR ((icd10_code)::text ~ '^[A-Z][0-9]{2}(\.[0-9A-Z]{1,4})?$'::text)))
ck_consultation_diagnoses_line	CHECK	CHECK (((line_number >= 1) AND (line_number <= 10)))
ck_consultation_diagnoses_text_not_blank	CHECK	CHECK ((btrim((condition_text)::text) <> ''::text))
fk_consultation_diagnoses_consultation	FOREIGN KEY	FOREIGN KEY (consultation_id) REFERENCES consultations(id)
pk_consultation_diagnoses	PRIMARY KEY	PRIMARY KEY (id)
uq_consultation_diagnoses_line	UNIQUE	UNIQUE (consultation_id, line_number)

4.9 medicines

Purpose. Optional catalogue used for prescription autocomplete and to block discontinued (inactive) medicines. Contains no stock levels: inventory is out of scope for v1.

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK	
name	character varying(200)	NOT NULL	-	-	Generic (or brand) name, e.g. "Paracetamol".
strength	character varying(50)	null	-	-	Optional strength, e.g. "500 mg" or "125 mg/5 ml".
dosage_form	character varying(20)	NOT NULL	-	-	Dosage form enumeration.
active	boolean	NOT NULL	true	-	FALSE = discontinued/unavailable; cannot be prescribed by medicineId (422 MEDICINE_INACTIVE).
created_at	timestamp with time zone	NOT NULL	now()	-	
created_by	uuid	NOT NULL	-	FK → users	
updated_at	timestamp with time zone	NOT NULL	now()	-	
updated_by	uuid	NOT NULL	-	FK → users	
version	bigint	NOT NULL	0	-	

Constraints and indexes

Name	Kind	Definition
`ck_medicines_dosage_form`	CHECK	`CHECK (((dosage_form)::text = ANY ((ARRAY['TABLET'::character varying, 'CAPSULE'::character varying, 'SYRUP'::character varying, 'SUSPENSION'::character varying, 'SOLUTION'::character varying, 'INJECTION'::character varying, 'CREAM'::character varying, 'OINTMENT'::character varying, 'GEL'::character varying, 'DROPS'::character varying, 'INHALER'::character varying, 'SUPPOSITORY'::character varying, 'POWDER'::character varying, 'PATCH'::character varying, 'OTHER'::character varying])::text[])))`
`ck_medicines_name_not_blank`	CHECK	`CHECK ((btrim((name)::text) <> ''::text))`
`fk_medicines_created_by`	FOREIGN KEY	`FOREIGN KEY (created_by) REFERENCES users(id)`
`fk_medicines_updated_by`	FOREIGN KEY	`FOREIGN KEY (updated_by) REFERENCES users(id)`
`pk_medicines`	PRIMARY KEY	`PRIMARY KEY (id)`
`ix_medicines_name_trgm`	INDEX	`gin (name gin_trgm_ops)`
`uq_medicines_name_strength_form`	UNIQUE INDEX	`btree (lower((name)::text), lower((COALESCE(strength, ''::character varying))::text), dosage_form)`

Referenced by: `prescription_items`.

4.10 prescriptions

Purpose. Prescription header. Lifecycle PENDING → IN_PROGRESS → DISPENSED, with CANCELLED from PENDING/IN_PROGRESS. DISPENSED and CANCELLED are terminal.

Column	Type	Null	Default	Keys	Description
`id`	uuid	NOT NULL	gen_random_uuid()	PK	
`prescription_number`	bigint	NOT NULL	nextval('prescription_number_seq'::regclass)	UK	Human-readable sequential number, shown as RX-000123 (format RX-%06d). Gaps are possible and harmless.
`consultation_id`	uuid	NOT NULL	-	FK → consultations	Consultation that produced the prescription. The prescriber is consultations.doctor_id.
`patient_id`	uuid	NOT NULL	-	FK → consultations FK → patients	Deliberate denormalisation for the pharmacist queue; kept consistent with the consultation by a composite foreign key.

Column	Type	Null	Default	Keys	Description
status	character varying(12)	NOT NULL	'PENDING'	-	PENDING | IN_PROGRESS | DISPENSED | CANCELLED.
notes	character varying(1000)	null	-	-	Optional note from doctor to pharmacist.
submitted_at	timestamp with time zone	NOT NULL	now()	-	Instant the prescription entered the pharmacist queue.
started_at	timestamp with time zone	null	-	-	Instant a pharmacist marked it IN_PROGRESS.
started_by	uuid	null	-	FK → users	Pharmacist who marked it IN_PROGRESS.
cancelled_at	timestamp with time zone	null	-	-	Cancellation instant.
cancelled_by	uuid	null	-	FK → users	Doctor (prescriber) or pharmacist who cancelled.
cancel_reason	character varying(500)	null	-	-	Mandatory reason for cancellation.
supersedes_prescription_id	uuid	null	-	FK → prescriptions UK	Cancelled prescription that this one replaces (re-issue); each prescription can be superseded at most once.
updated_at	timestamp with time zone	NOT NULL	now()	-	
version	bigint	NOT NULL	0	-	Optimistic-locking version.

Constraints and indexes

Name	Kind	Definition
ck_prescriptions_not_self_superseding	CHECK	CHECK (((supersedes_prescription_id IS NULL) OR (supersedes_prescription_id <> id)))
ck_prescriptions_status	CHECK	CHECK (((status)::text = ANY ((ARRAY['PENDING'::character varying, 'IN_PROGRESS'::character varying, 'DISPENSED'::character varying, 'CANCELLED'::character varying])::text[])))
ck_prescriptions_status_fields	CHECK	CHECK (((((status)::text <> 'IN_PROGRESS'::text) OR (started_at IS NOT NULL)) AND (((status)::text <> 'CANCELLED'::text) OR ((cancelled_at IS NOT NULL) AND (cancelled_by IS NOT NULL) AND (cancel_reason IS NOT NULL))) AND (((status)::text = 'CANCELLED'::text) OR (cancelled_at IS NULL))))
fk_prescriptions_cancelled_by	FOREIGN KEY	FOREIGN KEY (cancelled_by) REFERENCES users(id)

Name	Kind	Definition
fk_prescriptions_consultation_patient	FOREIGN KEY	FOREIGN KEY (consultation_id, patient_id) REFERENCES consultations(id, patient_id)
fk_prescriptions_patient	FOREIGN KEY	FOREIGN KEY (patient_id) REFERENCES patients(id)
fk_prescriptions_started_by	FOREIGN KEY	FOREIGN KEY (started_by) REFERENCES users(id)
fk_prescriptions_supersedes	FOREIGN KEY	FOREIGN KEY (supersedes_prescription_id) REFERENCES prescriptions(id)
pk_prescriptions	PRIMARY KEY	PRIMARY KEY (id)
uq_prescriptions_number	UNIQUE	UNIQUE (prescription_number)
uq_prescriptions_supersedes	UNIQUE	UNIQUE (supersedes_prescription_id)
ix_prescriptions_consultation	INDEX	btree (consultation_id)
ix_prescriptions_patient_submitted	INDEX	btree (patient_id, submitted_at DESC)
ix_prescriptions_status_submitted	INDEX	btree (status, submitted_at)
uq_prescriptions_active_per_consultation	UNIQUE INDEX	btree (consultation_id) WHERE ((status)::text <> 'CANCELLED'::text)

Referenced by: `dispensings`, `prescription_items`.

4.11 prescription_items

Purpose. Prescribed lines (1-30 per prescription). medicine_name/strength/dosage_form are snapshots so history is unchanged if the catalogue is edited later.

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK UK (composite)	
prescription_id	uuid	NOT NULL	-	FK → prescriptions UK (composite)	
line_number	smallint	NOT NULL	-	UK (composite)	Order within the prescription.
medicine_id	uuid	null	-	FK → medicines	Catalogue medicine, or NULL for a free-text medicine.
medicine_name	character varying(200)	NOT NULL	-	-	Snapshot of the medicine name at prescribing time.

Column	Type	Null	Default	Keys	Description
strength	character varying(50)	null	-	-	Snapshot of strength, e.g. "500 mg".
dosage_form	character varying(20)	null	-	-	Snapshot of dosage form.
dose	character varying(100)	NOT NULL	-	-	Dose per administration, free text, e.g. "1 tablet", "5 ml".
frequency	character varying(100)	NOT NULL	-	-	Frequency, free text, e.g. "3 times daily (TDS)".
duration_days	smallint	null	-	-	Duration in days (1-365); NULL = as directed / as needed.
quantity	numeric(7,2)	NOT NULL	-	-	Total quantity to dispense (BigDecimal, max 2 decimals).
quantity_unit	character varying(30)	NOT NULL	-	-	Unit of quantity, e.g. "tablets", "ml", "bottles".
instructions	character varying(500)	null	-	-	Optional patient instructions, e.g. "after meals".

Constraints and indexes

Name	Kind	Definition
ck_prescription_items_dosage_form	CHECK	CHECK (((dosage_form IS NULL) OR ((dosage_form)::text = ANY ((ARRAY['TABLET'::character varying, 'CAPSULE'::character varying, 'SYRUP'::character varying, 'SUSPENSION'::character varying, 'SOLUTION'::character varying, 'INJECTION'::character varying, 'CREAM'::character varying, 'OINTMENT'::character varying, 'GEL'::character varying, 'DROPS'::character varying, 'INHALER'::character varying, 'SUPPOSITORY'::character varying, 'POWDER'::character varying, 'PATCH'::character varying, 'OTHER'::character varying])::text[]))))
ck_prescription_items_duration	CHECK	CHECK (((duration_days IS NULL) OR ((duration_days >= 1) AND (duration_days <= 365))))
ck_prescription_items_line	CHECK	CHECK (((line_number >= 1) AND (line_number <= 30)))
ck_prescription_items_name_not_blank	CHECK	CHECK ((btrim((medicine_name)::text) <> ''::text))
ck_prescription_items_quantity	CHECK	CHECK ((quantity > (0)::numeric))
fk_prescription_items_medicine	FOREIGN KEY	FOREIGN KEY (medicine_id) REFERENCES medicines(id)
fk_prescription_items_prescription	FOREIGN KEY	FOREIGN KEY (prescription_id) REFERENCES prescriptions(id)
pk_prescription_items	PRIMARY KEY	PRIMARY KEY (id)
uq_prescription_items_id_prescription	UNIQUE	UNIQUE (id, prescription_id)
uq_prescription_items_line	UNIQUE	UNIQUE (prescription_id, line_number)

Referenced by: `dispensing_items`.

4.12 dispensings

Purpose. Who dispensed, when, and the overall outcome. UNIQUE(prescription_id) makes duplicate dispensing impossible at the database level.

Column	Type	Null	Default	Keys	Description
`id`	uuid	NOT NULL	gen_random_uuid()	PK UK (composite)	
`prescription_id`	uuid	NOT NULL	-	FK → prescriptions UK (composite) UK	
`pharmacist_id`	uuid	NOT NULL	-	FK → users	Authenticated pharmacist who confirmed dispensing.
`dispensed_at`	timestamp with time zone	NOT NULL	now()	-	Server instant of confirmation.
`outcome`	character varying(10)	NOT NULL	-	-	FULL (every item fully dispensed) | PARTIAL (at least one item partial or not dispensed). Derived by the server.
`notes`	character varying(1000)	null	-	-	Optional pharmacist note.

Constraints and indexes

Name	Kind	Definition
`ck_dispensings_outcome`	CHECK	`CHECK (((outcome)::text = ANY ((ARRAY['FULL'::character varying, 'PARTIAL'::character varying])::text[])))`
`fk_dispensings_pharmacist`	FOREIGN KEY	`FOREIGN KEY (pharmacist_id) REFERENCES users(id)`
`fk_dispensings_prescription`	FOREIGN KEY	`FOREIGN KEY (prescription_id) REFERENCES prescriptions(id)`
`pk_dispensings`	PRIMARY KEY	`PRIMARY KEY (id)`
`uq_dispensings_id_prescription`	UNIQUE	`UNIQUE (id, prescription_id)`
`uq_dispensings_prescription`	UNIQUE	`UNIQUE (prescription_id)`
`ix_dispensings_pharmacist_at`	INDEX	`btree (pharmacist_id, dispensed_at DESC)`

Referenced by: `dispensing_items`.

4.13 dispensing_items

Purpose. One row per prescription item in the dispensing. Composite foreign keys guarantee the item belongs to the same prescription as the dispensing.

Column	Type	Null	Default	Keys	Description
`id`	uuid	NOT NULL	gen_random_uuid()	PK	
`dispensing_id`	uuid	NOT NULL	-	FK → dispensings	
`prescription_id`	uuid	NOT NULL	-	FK → dispensings FK → prescription_items	Redundant copy used only by the composite foreign keys.
`prescription_item_id`	uuid	NOT NULL	-	FK → prescription_items UK	
`quantity_dispensed`	numeric(7,2)	NOT NULL	-	-	Quantity handed over (0 .. prescribed quantity, max 2 decimals).
`status`	character varying(15)	NOT NULL	-	-	FULL | PARTIAL | NOT_DISPENSED, derived by the server from quantity_dispensed vs prescribed quantity.
`note`	character varying(500)	null	-	-	Mandatory when not FULL, e.g. "out of stock", "brand substituted".

Constraints and indexes

Name	Kind	Definition
`ck_dispensing_items_note_required`	CHECK	`CHECK ((((status)::text = 'FULL'::text) OR (note IS NOT NULL)))`
`ck_dispensing_items_quantity`	CHECK	`CHECK ((quantity_dispensed >= (0)::numeric))`
`ck_dispensing_items_status`	CHECK	`CHECK (((status)::text = ANY ((ARRAY['FULL'::character varying, 'PARTIAL'::character varying, 'NOT_DISPENSED'::character varying])::text[])))`
`ck_dispensing_items_zero_iff_not_dispensed`	CHECK	`CHECK ((((status)::text = 'NOT_DISPENSED'::text) = (quantity_dispensed = (0)::numeric)))`
`fk_dispensing_items_dispensing`	FOREIGN KEY	`FOREIGN KEY (dispensing_id, prescription_id) REFERENCES dispensings(id, prescription_id)`
`fk_dispensing_items_item`	FOREIGN KEY	`FOREIGN KEY (prescription_item_id, prescription_id) REFERENCES prescription_items(id, prescription_id)`

Name	Kind	Definition
pk_dispensing_items	PRIMARY KEY	PRIMARY KEY (id)
uq_dispensing_items_item	UNIQUE	UNIQUE (prescription_item_id)
ix_dispensing_items_dispensing	INDEX	btree (dispensing_id)

4.14 idempotency_keys

Purpose. Inserted in the SAME transaction as the business write. A concurrent duplicate blocks on the unique index until the first commits, then replays the stored resource. Stores ids only, never response bodies (no PHI copies).

Column	Type	Null	Default	Keys	Description
id	uuid	NOT NULL	gen_random_uuid()	PK	
user_id	uuid	NOT NULL	-	FK → users UK (composite)	
idempotency_key	character varying(100)	NOT NULL	-	UK (composite)	Client-supplied Idempotency-Key header (UUID recommended).
operation	character varying(50)	NOT NULL	-	-	Operation id, e.g. createConsultation. Same key with a different operation or body → 422 IDEMPOTENCY_KEY_REUSED.
request_hash	character(64)	NOT NULL	-	-	SHA-256 of operation + path + canonical JSON body.
resource_id	uuid	null	-	-	Id of the created resource, used to rebuild the response on replay.
response_status	smallint	null	-	-	HTTP status of the original response (201 or 200).
created_at	timestamp with time zone	NOT NULL	now()	-	
expires_at	timestamp with time zone	NOT NULL	-	-	created_at + 24 h; purged by the retention job.

Constraints and indexes

Name	Kind	Definition
ck_idempotency_keys_hash_hex	CHECK	CHECK ((request_hash ~ '^[0-9a-f]{64}$'::text))
ck_idempotency_keys_key_format	CHECK	CHECK (((idempotency_key)::text ~ '^[A-Za-z0-9_-]{8,100}$'::text))

Name	Kind	Definition
`fk_idempotency_keys_user`	FOREIGN KEY	`FOREIGN KEY (user_id) REFERENCES users(id)`
`pk_idempotency_keys`	PRIMARY KEY	`PRIMARY KEY (id)`
`uq_idempotency_keys_user_key`	UNIQUE	`UNIQUE (user_id, idempotency_key)`
`ix_idempotency_keys_expires_at`	INDEX	`btree (expires_at)`

4.15 audit_events

Purpose. Append-only audit trail (UPDATE/DELETE blocked by trigger). details must never contain passwords, tokens, clinical notes or diagnoses.

Column	Type	Null	Default	Keys	Description
`id`	uuid	NOT NULL	gen_random_uuid()	PK	
`occurred_at`	timestamp with time zone	NOT NULL	now()	-	Event instant (UTC).
`actor_user_id`	uuid	null	-	FK → users	Authenticated user, NULL for anonymous events (e.g. failed login of unknown user).
`actor_username`	character varying(50)	null	-	-	Username snapshot (only for existing users).
`action`	character varying(50)	NOT NULL	-	-	AuditAction enumeration value, e.g. PATIENT_HISTORY_VIEWED.
`entity_type`	character varying(30)	NOT NULL	-	-	Type of the affected entity.
`entity_id`	uuid	null	-	-	Id of the affected entity.
`patient_id`	uuid	null	-	FK → patients	Patient whose data was touched; supports "who accessed this patient" reports.
`outcome`	character varying(10)	NOT NULL	'SUCCESS'	-	SUCCESS | FAILURE | DENIED.
`details`	jsonb	NOT NULL	'{}'	-	Small JSON object with non-sensitive context (e.g. {"fromStatus":"PENDING","toStatus":"DISPENSED"}).
`request_id`	character varying(64)	null	-	-	X-Request-Id correlating with application logs.
`ip_address`	inet	null	-	-	Client IP (from the trusted proxy).
`user_agent`	character varying(255)	null	-	-	Truncated User-Agent.

Constraints and indexes

Name	Kind	Definition
ck_audit_events_action_format	CHECK	CHECK (((action)::text ~ '^[A-Z][A-Z_]{2,49}$'::text))
ck_audit_events_details_object	CHECK	CHECK ((jsonb_typeof(details) = 'object'::text))
ck_audit_events_entity_type	CHECK	CHECK (((entity_type)::text = ANY ((ARRAY['USER'::character varying, 'SESSION'::character varying, 'PATIENT'::character varying, 'CONSULTATION'::character varying, 'PRESCRIPTION'::character varying, 'MEDICINE'::character varying, 'QUEUE_SESSION'::character varying, 'QUEUE_ENTRY'::character varying, 'AUDIT'::character varying])::text[])))
ck_audit_events_outcome	CHECK	CHECK (((outcome)::text = ANY ((ARRAY['SUCCESS'::character varying, 'FAILURE'::character varying, 'DENIED'::character varying])::text[])))
fk_audit_events_actor	FOREIGN KEY	FOREIGN KEY (actor_user_id) REFERENCES users(id)
fk_audit_events_patient	FOREIGN KEY	FOREIGN KEY (patient_id) REFERENCES patients(id)
pk_audit_events	PRIMARY KEY	PRIMARY KEY (id)
ix_audit_events_actor	INDEX	btree (actor_user_id, occurred_at DESC)
ix_audit_events_entity	INDEX	btree (entity_type, entity_id)
ix_audit_events_occurred_at	INDEX	btree (occurred_at DESC)
ix_audit_events_patient	INDEX	btree (patient_id, occurred_at DESC) WHERE (patient_id IS NOT NULL)

5. SQL DDL

The complete schema is a single Flyway migration, `backend/src/main/resources/db/migration/V1__initial_schema.sql`. It was executed successfully against PostgreSQL 16.15 together with the negative constraint tests and a 60-way concurrent booking test (section 7.6).

```
-- ============================================================================
-- Dispensary Management System (DMS) - initial schema
-- Flyway migration: V1__initial_schema.sql
-- Target: PostgreSQL 16+ (works on 14+)
-- Conventions:
--   * Primary keys are UUID (gen_random_uuid(), core since PG 13).
--     The application normally generates the UUID; the DB default
--     exists for seed scripts and manual inserts.
--   * All instants are TIMESTAMPTZ and stored in UTC.
--   * Business dates (session_date, date_of_birth) are DATE and are
--     interpreted in the dispensary time zone (APP_TIMEZONE).
--   * Enumerations are VARCHAR + CHECK constraints (not PG ENUM types)
--     so new values can be added with a simple constraint change.
--   * Constraint and index names are stable: the backend maps unique
--     constraint names to API error codes (see ErrorCode catalogue).
-- ============================================================================

CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- ----------------------------------------------------------------------------
-- users: staff accounts (doctors, pharmacists, administrators)
-- ----------------------------------------------------------------------------
CREATE TABLE users (
    id                              UUID         NOT NULL DEFAULT gen_random_uuid(),
    username                        VARCHAR(50)  NOT NULL,
    password_hash                   VARCHAR(255) NOT NULL,
    full_name                       VARCHAR(150) NOT NULL,
    professional_registration_number VARCHAR(50),
    enabled                         BOOLEAN      NOT NULL DEFAULT TRUE,
    must_change_password            BOOLEAN      NOT NULL DEFAULT TRUE,
    failed_login_count              INTEGER      NOT NULL DEFAULT 0,
    locked_until                    TIMESTAMPTZ,
    last_login_at                   TIMESTAMPTZ,
    password_changed_at             TIMESTAMPTZ,
    created_at                      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    created_by                      UUID,
    updated_at                      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    version                         BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT pk_users PRIMARY KEY (id),
    CONSTRAINT uq_users_username UNIQUE (username),
    CONSTRAINT ck_users_username_format CHECK (username ~ '^[a-z0-9._-]{3,50}$'),
    CONSTRAINT ck_users_full_name_not_blank CHECK (btrim(full_name) <> ''),
    CONSTRAINT ck_users_failed_login_count CHECK (failed_login_count >= 0),
    CONSTRAINT fk_users_created_by FOREIGN KEY (created_by) REFERENCES users (id)
);
COMMENT ON TABLE users IS 'Staff accounts that can authenticate. Users are never hard-deleted; they are
disabled (enabled = false) so that historical references stay valid.';
COMMENT ON COLUMN users.id IS 'Surrogate key (UUID).';
COMMENT ON COLUMN users.username IS 'Login name, stored lower-case. Pattern ^[a-z0-9._-]{3,50}$.';
COMMENT ON COLUMN users.password_hash IS 'Spring Security DelegatingPasswordEncoder value, e.g. {bcrypt}
$2b$12$... Never returned by the API.';
COMMENT ON COLUMN users.full_name IS 'Display name shown on prescriptions and audit views.';
COMMENT ON COLUMN users.professional_registration_number IS 'Optional professional registration number
(e.g. medical council or pharmacy council number).';
COMMENT ON COLUMN users.enabled IS 'FALSE = account disabled; login and token refresh are refused.';
COMMENT ON COLUMN users.must_change_password IS 'TRUE after account creation or admin password reset; all
endpoints except auth/me, auth/change-password and auth/logout return 403 PASSWORD_CHANGE_REQUIRED.';
COMMENT ON COLUMN users.failed_login_count IS 'Consecutive failed logins; reset to 0 on success.';
COMMENT ON COLUMN users.locked_until IS 'Login refused until this instant (set after 5 consecutive
failures, 15 minutes).';
COMMENT ON COLUMN users.last_login_at IS 'Instant of the last successful login.';
COMMENT ON COLUMN users.password_changed_at IS 'Instant the password was last set by the user.';
COMMENT ON COLUMN users.created_at IS 'Creation instant (UTC).';
COMMENT ON COLUMN users.created_by IS 'Admin who created the account; NULL for the bootstrap admin.';
COMMENT ON COLUMN users.updated_at IS 'Last modification instant (UTC).';
COMMENT ON COLUMN users.version IS 'Optimistic-locking version (JPA @Version).';

-- ----------------------------------------------------------------------------
```

```sql
-- user_roles: roles granted to a user (a user may hold several roles)
-- ------------------------------------------------------------------------
CREATE TABLE user_roles (
    user_id UUID        NOT NULL,
    role    VARCHAR(20) NOT NULL,
    CONSTRAINT pk_user_roles PRIMARY KEY (user_id, role),
    CONSTRAINT ck_user_roles_role CHECK (role IN ('ADMIN', 'DOCTOR', 'PHARMACIST')),
    CONSTRAINT fk_user_roles_user FOREIGN KEY (user_id) REFERENCES users (id) ON DELETE CASCADE
);
COMMENT ON TABLE user_roles IS 'Role grants. Roles are a fixed code-level enumeration, so no separate
roles table is needed.';
COMMENT ON COLUMN user_roles.user_id IS 'Granted user.';
COMMENT ON COLUMN user_roles.role IS 'ADMIN | DOCTOR | PHARMACIST.';
CREATE INDEX ix_user_roles_role ON user_roles (role);

-- ------------------------------------------------------------------------
-- refresh_tokens: server-side record of issued refresh tokens
-- ------------------------------------------------------------------------
CREATE TABLE refresh_tokens (
    id               UUID         NOT NULL DEFAULT gen_random_uuid(),
    user_id          UUID         NOT NULL,
    family_id        UUID         NOT NULL,
    token_hash       CHAR(64)     NOT NULL,
    issued_at        TIMESTAMPTZ  NOT NULL DEFAULT now(),
    expires_at       TIMESTAMPTZ  NOT NULL,
    rotated_at       TIMESTAMPTZ,
    replaced_by_id   UUID,
    revoked_at       TIMESTAMPTZ,
    revoke_reason    VARCHAR(30),
    created_ip       INET,
    user_agent       VARCHAR(255),
    CONSTRAINT pk_refresh_tokens PRIMARY KEY (id),
    CONSTRAINT uq_refresh_tokens_token_hash UNIQUE (token_hash),
    CONSTRAINT ck_refresh_tokens_hash_hex CHECK (token_hash ~ '^[0-9a-f]{64}$'),
    CONSTRAINT ck_refresh_tokens_expiry CHECK (expires_at > issued_at),
    CONSTRAINT ck_refresh_tokens_revoke_reason CHECK (
        revoke_reason IS NULL OR revoke_reason IN
        ('LOGOUT', 'REUSE_DETECTED', 'PASSWORD_CHANGED', 'PASSWORD_RESET', 'USER_DISABLED')),
    CONSTRAINT ck_refresh_tokens_revoked_consistency CHECK ((revoked_at IS NULL) = (revoke_reason IS NULL)
),
    CONSTRAINT fk_refresh_tokens_user FOREIGN KEY (user_id) REFERENCES users (id),
    CONSTRAINT fk_refresh_tokens_replaced_by FOREIGN KEY (replaced_by_id) REFERENCES refresh_tokens (id)
);
COMMENT ON TABLE refresh_tokens IS 'Opaque refresh tokens (only the SHA-256 hash is stored). Rotated on
every use; reuse of a rotated token outside the grace window revokes the whole family.';
COMMENT ON COLUMN refresh_tokens.family_id IS 'All tokens descending from one login share a family; the
family expires APP_JWT_REFRESH_TOKEN_TTL after login.';
COMMENT ON COLUMN refresh_tokens.token_hash IS 'Lower-case hex SHA-256 of the raw token. The raw token
exists only in the HttpOnly cookie.';
COMMENT ON COLUMN refresh_tokens.issued_at IS 'Issue instant.';
COMMENT ON COLUMN refresh_tokens.expires_at IS
'Absolute expiry; equals the family expiry (login instant + refresh TTL).';
COMMENT ON COLUMN refresh_tokens.rotated_at IS 'Set when the token was exchanged for a successor.';
COMMENT ON COLUMN refresh_tokens.replaced_by_id IS 'Successor token after rotation.';
COMMENT ON COLUMN refresh_tokens.revoked_at IS 'Set when revoked; revoked tokens are never accepted.';
COMMENT ON COLUMN refresh_tokens.revoke_reason IS 'LOGOUT | REUSE_DETECTED | PASSWORD_CHANGED |
PASSWORD_RESET | USER_DISABLED.';
COMMENT ON COLUMN refresh_tokens.created_ip IS 'Client IP at issue (for security investigation).';
COMMENT ON COLUMN refresh_tokens.user_agent IS 'Truncated User-Agent at issue.';
CREATE INDEX ix_refresh_tokens_user ON refresh_tokens (user_id);
CREATE INDEX ix_refresh_tokens_family ON refresh_tokens (family_id);
CREATE INDEX ix_refresh_tokens_expires_at ON refresh_tokens (expires_at);

-- ------------------------------------------------------------------------
-- patients: demographic master record
-- ------------------------------------------------------------------------
CREATE TABLE patients (
    id                      UUID          NOT NULL DEFAULT gen_random_uuid(),
    patient_number          VARCHAR(20)   NOT NULL,
    full_name               VARCHAR(150)  NOT NULL,
    sex                     VARCHAR(10)   NOT NULL,
    date_of_birth           DATE          NOT NULL,
    date_of_birth_estimated BOOLEAN       NOT NULL DEFAULT FALSE,
    phone_number            VARCHAR(16),
    address                 VARCHAR(300),
    known_allergies         VARCHAR(1000),
    created_at              TIMESTAMPTZ   NOT NULL DEFAULT now(),
    created_by              UUID          NOT NULL,
    updated_at              TIMESTAMPTZ   NOT NULL DEFAULT now(),
```

```
    updated_by              UUID          NOT NULL,
    version                 BIGINT        NOT NULL DEFAULT 0,
    CONSTRAINT pk_patients PRIMARY KEY (id),
    CONSTRAINT uq_patients_patient_number UNIQUE (patient_number),
    CONSTRAINT ck_patients_patient_number_format CHECK (patient_number ~ '^[A-Z0-9][A-Z0-9-]{2,19}$'),
    CONSTRAINT ck_patients_full_name_not_blank CHECK (btrim(full_name) <> ''),
    CONSTRAINT ck_patients_sex CHECK (sex IN ('MALE', 'FEMALE', 'OTHER', 'UNKNOWN')),
    CONSTRAINT ck_patients_dob_min CHECK (date_of_birth >= DATE '1900-01-01'),
    CONSTRAINT ck_patients_phone_format CHECK (phone_number IS NULL OR phone_number ~ '^\+?[0-9]{7,15}$'),
    CONSTRAINT fk_patients_created_by FOREIGN KEY (created_by) REFERENCES users (id),
    CONSTRAINT fk_patients_updated_by FOREIGN KEY (updated_by) REFERENCES users (id)
);
COMMENT ON TABLE patients IS 'One row per patient. Clinical history is NOT stored here; it is the set of
consultations that reference the patient.';
COMMENT ON COLUMN patients.patient_number IS 'Business identifier printed on the clinic card; entered by
staff, normalised to upper case, immutable after registration.';
COMMENT ON COLUMN patients.full_name IS 'Patient full name.';
COMMENT ON COLUMN patients.sex IS 'MALE | FEMALE | OTHER | UNKNOWN.';
COMMENT ON COLUMN patients.date_of_birth IS 'Exact DOB, or a DOB derived from an estimated age (see
date_of_birth_estimated). Age is always computed, never stored.';
COMMENT ON COLUMN patients.date_of_birth_estimated IS
'TRUE when the DOB was derived from an estimated age because the real DOB is unknown.';
COMMENT ON COLUMN patients.phone_number IS 'Optional contact number, digits with optional leading +, 7-15
digits.';
COMMENT ON COLUMN patients.address IS 'Optional postal address.';
COMMENT ON COLUMN patients.known_allergies IS 'Clinician-entered allergy statement shown to doctors and
pharmacists. Changes are audited.';
COMMENT ON COLUMN patients.created_by IS 'Doctor who registered the patient.';
COMMENT ON COLUMN patients.updated_by IS 'Last user who changed demographics.';
COMMENT ON COLUMN patients.version IS 'Optimistic-locking version; PUT requests must send the current
value.';
CREATE INDEX ix_patients_full_name_trgm ON patients USING gin (full_name gin_trgm_ops);
CREATE INDEX ix_patients_phone_number ON patients (phone_number) WHERE phone_number IS NOT NULL;

-- ---------------------------------------------------------------------
-- queue_sessions: one numbered queue (e.g. a doctor's clinic on a date)
-- ---------------------------------------------------------------------
CREATE TABLE queue_sessions (
    id                  UUID         NOT NULL DEFAULT gen_random_uuid(),
    session_date        DATE         NOT NULL,
    name                VARCHAR(100) NOT NULL,
    assigned_doctor_id  UUID,
    status              VARCHAR(12)  NOT NULL DEFAULT 'SCHEDULED',
    last_issued_number  INTEGER      NOT NULL DEFAULT 0,
    last_called_number  INTEGER,
    opened_at           TIMESTAMPTZ,
    opened_by           UUID,
    closed_at           TIMESTAMPTZ,
    closed_by           UUID,
    created_at          TIMESTAMPTZ  NOT NULL DEFAULT now(),
    created_by          UUID         NOT NULL,
    updated_at          TIMESTAMPTZ  NOT NULL DEFAULT now(),
    version             BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT pk_queue_sessions PRIMARY KEY (id),
    CONSTRAINT ck_queue_sessions_name_not_blank CHECK (btrim(name) <> ''),
    CONSTRAINT ck_queue_sessions_status CHECK (status IN ('SCHEDULED', 'OPEN', 'CLOSED')),
    CONSTRAINT ck_queue_sessions_last_issued CHECK (last_issued_number BETWEEN 0 AND 999),
    CONSTRAINT ck_queue_sessions_last_called CHECK (
        last_called_number IS NULL OR (last_called_number BETWEEN 1 AND last_issued_number)),
    CONSTRAINT ck_queue_sessions_status_timestamps CHECK (
        (status = 'SCHEDULED' AND opened_at IS NULL AND closed_at IS NULL)
     OR (status = 'OPEN'      AND opened_at IS NOT NULL AND closed_at IS NULL)
     OR (status = 'CLOSED'    AND closed_at IS NOT NULL)),
    CONSTRAINT fk_queue_sessions_doctor FOREIGN KEY (assigned_doctor_id) REFERENCES users (id),
    CONSTRAINT fk_queue_sessions_opened_by FOREIGN KEY (opened_by) REFERENCES users (id),
    CONSTRAINT fk_queue_sessions_closed_by FOREIGN KEY (closed_by) REFERENCES users (id),
    CONSTRAINT fk_queue_sessions_created_by FOREIGN KEY (created_by) REFERENCES users (id)
);
COMMENT ON TABLE queue_sessions IS 'A queue with its own number sequence. Numbering restarts at 1 for
every session; sessions are per date, so numbering restarts at least daily.';
COMMENT ON COLUMN queue_sessions.session_date IS 'Business date of the session in APP_TIMEZONE.';
COMMENT ON COLUMN queue_sessions.name IS 'Display label, e.g. "Morning OPD - Room 1". Unique per date
(case-insensitive).';
COMMENT ON COLUMN queue_sessions.assigned_doctor_id IS 'Optional doctor who normally serves this queue
(informational; any doctor may call).';
COMMENT ON COLUMN queue_sessions.status IS 'SCHEDULED (bookings allowed, no calls) | OPEN (bookings and
calls) | CLOSED (terminal).';
COMMENT ON COLUMN queue_sessions.last_issued_number IS
'Counter of issued queue numbers. Incremented under a row lock in the same transaction that inserts the
```

```
queue entry, so numbers are unique and gap-free.';
COMMENT ON COLUMN queue_sessions.last_called_number IS 'Number most recently called (the public "now
serving" number). Used as the compare-and-set token for call-next.';
COMMENT ON COLUMN queue_sessions.opened_at IS 'Instant the session was opened.';
COMMENT ON COLUMN queue_sessions.closed_at IS 'Instant the session was closed.';
COMMENT ON COLUMN queue_sessions.version IS
'Incremented on every queue mutation (booking, call, complete, skip, cancel, requeue); exposed as the
public ETag.';
CREATE UNIQUE INDEX uq_queue_sessions_date_name ON queue_sessions (session_date, lower(name));
CREATE INDEX ix_queue_sessions_date_status ON queue_sessions (session_date, status);

-- ---------------------------------------------------------------------
-- queue_entries: a booking / position in a queue session
-- ---------------------------------------------------------------------
CREATE TABLE queue_entries (
    id                UUID         NOT NULL DEFAULT gen_random_uuid(),
    queue_session_id  UUID         NOT NULL,
    queue_number      INTEGER      NOT NULL,
    status            VARCHAR(12)  NOT NULL DEFAULT 'WAITING',
    channel           VARCHAR(10)  NOT NULL,
    patient_name      VARCHAR(150),
    contact_phone     VARCHAR(16),
    notes             VARCHAR(300),
    booked_at         TIMESTAMPTZ  NOT NULL DEFAULT now(),
    booked_by         UUID         NOT NULL,
    called_at         TIMESTAMPTZ,
    called_by         UUID,
    completed_at      TIMESTAMPTZ,
    completed_by      UUID,
    skipped_at        TIMESTAMPTZ,
    skipped_by        UUID,
    skip_reason       VARCHAR(300),
    cancelled_at      TIMESTAMPTZ,
    cancelled_by      UUID,
    cancel_reason     VARCHAR(500),
    requeue_count     SMALLINT     NOT NULL DEFAULT 0,
    anonymized_at     TIMESTAMPTZ,
    updated_at        TIMESTAMPTZ  NOT NULL DEFAULT now(),
    version           BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT pk_queue_entries PRIMARY KEY (id),
    CONSTRAINT uq_queue_entries_session_number UNIQUE (queue_session_id, queue_number),
    CONSTRAINT ck_queue_entries_number CHECK (queue_number BETWEEN 1 AND 999),
    CONSTRAINT ck_queue_entries_status CHECK (status IN ('WAITING', 'SERVING', 'COMPLETED', 'SKIPPED', 'CA
NCELLED')),
    CONSTRAINT ck_queue_entries_channel CHECK (channel IN ('PHONE', 'WALK_IN')),
    CONSTRAINT ck_queue_entries_name_present CHECK (patient_name IS NOT NULL OR anonymized_at IS NOT NULL)
,
    CONSTRAINT ck_queue_entries_phone_format CHECK (contact_phone IS NULL OR contact_phone ~ '^\+?[0-9]
{7,15}$'),
    CONSTRAINT ck_queue_entries_requeue_count CHECK (requeue_count >= 0),
    CONSTRAINT ck_queue_entries_status_timestamps CHECK (
        (status <> 'SERVING'   OR called_at IS NOT NULL)
    AND (status <> 'COMPLETED' OR completed_at IS NOT NULL)
    AND (status <> 'SKIPPED'   OR skipped_at IS NOT NULL)
    AND (status <> 'CANCELLED' OR (cancelled_at IS NOT NULL AND cancel_reason IS NOT NULL))),
    CONSTRAINT fk_queue_entries_session FOREIGN KEY (queue_session_id) REFERENCES queue_sessions (id),
    CONSTRAINT fk_queue_entries_booked_by FOREIGN KEY (booked_by) REFERENCES users (id),
    CONSTRAINT fk_queue_entries_called_by FOREIGN KEY (called_by) REFERENCES users (id),
    CONSTRAINT fk_queue_entries_completed_by FOREIGN KEY (completed_by) REFERENCES users (id),
    CONSTRAINT fk_queue_entries_skipped_by FOREIGN KEY (skipped_by) REFERENCES users (id),
    CONSTRAINT fk_queue_entries_cancelled_by FOREIGN KEY (cancelled_by) REFERENCES users (id)
);
COMMENT ON TABLE queue_entries IS 'A booking. Creating a booking (telephone or walk-in) creates exactly
one queue entry, so bookings and queue entries share one table.';
COMMENT ON COLUMN queue_entries.queue_session_id IS 'Session the entry belongs to.';
COMMENT ON COLUMN queue_entries.queue_number IS 'Number given to the patient; unique and gap-free within
the session, never reused.';
COMMENT ON COLUMN queue_entries.status IS 'WAITING | SERVING | COMPLETED | SKIPPED | CANCELLED (queue
lifecycle, distinct from prescription statuses).';
COMMENT ON COLUMN queue_entries.channel IS 'PHONE (telephone booking) | WALK_IN.';
COMMENT ON COLUMN queue_entries.patient_name IS 'Name given by the caller. Staff-only; never exposed
publicly. Set to NULL by the retention job (anonymized_at).';
COMMENT ON COLUMN queue_entries.contact_phone IS 'Optional callback number. Staff-only; used for
duplicate-booking detection. Cleared by the retention job.';
COMMENT ON COLUMN queue_entries.notes IS 'Short non-clinical booking note (e.g. "elderly, needs
wheelchair"). Cleared by the retention job.';
COMMENT ON COLUMN queue_entries.booked_by IS 'Staff member who registered the booking.';
COMMENT ON COLUMN queue_entries.called_at IS 'Last time the entry was called (moved to SERVING).';
COMMENT ON COLUMN queue_entries.requeue_count IS 'Number of times a skipped entry was returned to
```

```
WAITING.';
COMMENT ON COLUMN queue_entries.anonymized_at IS 'Set when the retention job removed name, phone and
notes.';
COMMENT ON COLUMN queue_entries.version IS 'Optimistic-locking version.';
CREATE INDEX ix_queue_entries_session_status_number ON queue_entries (queue_session_id, status, queue_numb
er);
CREATE INDEX ix_queue_entries_active_phone ON queue_entries (queue_session_id, contact_phone)
    WHERE status IN ('WAITING', 'SERVING') AND contact_phone IS NOT NULL;
CREATE INDEX ix_queue_entries_booked_at ON queue_entries (booked_at) WHERE anonymized_at IS NULL;

-- ---------------------------------------------------------------------
-- consultations: one doctor-patient encounter (immutable once created)
-- ---------------------------------------------------------------------
CREATE TABLE consultations (
    id                  UUID          NOT NULL DEFAULT gen_random_uuid(),
    patient_id          UUID          NOT NULL,
    doctor_id           UUID          NOT NULL,
    queue_entry_id      UUID,
    consultation_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    presenting_complaint VARCHAR(1000),
    clinical_notes      VARCHAR(5000),
    created_at          TIMESTAMPTZ   NOT NULL DEFAULT now(),
    CONSTRAINT pk_consultations PRIMARY KEY (id),
    CONSTRAINT uq_consultations_id_patient UNIQUE (id, patient_id),
    CONSTRAINT uq_consultations_queue_entry UNIQUE (queue_entry_id),
    CONSTRAINT fk_consultations_patient FOREIGN KEY (patient_id) REFERENCES patients (id),
    CONSTRAINT fk_consultations_doctor FOREIGN KEY (doctor_id) REFERENCES users (id),
    CONSTRAINT fk_consultations_queue_entry FOREIGN KEY (queue_entry_id) REFERENCES queue_entries (id)
);
COMMENT ON TABLE consultations IS 'Medical history is the ordered set of consultations of a patient. Rows
are insert-only: no UPDATE or DELETE paths exist in v1.';
COMMENT ON COLUMN consultations.patient_id IS 'Patient seen.';
COMMENT ON COLUMN consultations.doctor_id IS 'Doctor who recorded the consultation (the authenticated
user).';
COMMENT ON COLUMN consultations.queue_entry_id IS
'Optional link to the queue entry that was being served; at most one consultation per entry.';
COMMENT ON COLUMN consultations.consultation_at IS 'Instant the consultation was submitted (server
clock).';
COMMENT ON COLUMN consultations.presenting_complaint IS 'Optional presenting complaint / reason for
visit.';
COMMENT ON COLUMN consultations.clinical_notes IS
'Optional clinical notes. Sensitive: never logged, never shown to pharmacists.';
COMMENT ON COLUMN consultations.created_at IS 'Row creation instant.';
CREATE INDEX ix_consultations_patient_at ON consultations (patient_id, consultation_at DESC);
CREATE INDEX ix_consultations_doctor_at ON consultations (doctor_id, consultation_at DESC);

-- ---------------------------------------------------------------------
-- consultation_diagnoses: conditions / diagnoses recorded per consultation
-- ---------------------------------------------------------------------
CREATE TABLE consultation_diagnoses (
    id              UUID         NOT NULL DEFAULT gen_random_uuid(),
    consultation_id UUID         NOT NULL,
    line_number     SMALLINT     NOT NULL,
    condition_text  VARCHAR(500) NOT NULL,
    icd10_code      VARCHAR(10),
    CONSTRAINT pk_consultation_diagnoses PRIMARY KEY (id),
    CONSTRAINT uq_consultation_diagnoses_line UNIQUE (consultation_id, line_number),
    CONSTRAINT ck_consultation_diagnoses_line CHECK (line_number BETWEEN 1 AND 10),
    CONSTRAINT ck_consultation_diagnoses_text_not_blank CHECK (btrim(condition_text) <> ''),
    CONSTRAINT ck_consultation_diagnoses_icd10 CHECK (icd10_code IS NULL OR icd10_code ~ '^[A-Z][0-9]{2}
(\.[0-9A-Z]{1,4})?$'),
    CONSTRAINT fk_consultation_diagnoses_consultation FOREIGN KEY (consultation_id) REFERENCES consultatio
ns (id)
);
COMMENT ON TABLE consultation_diagnoses IS 'One row per condition/diagnosis recorded in a consultation
(1-10 per consultation). "Previous medical conditions" are derived from these rows.';
COMMENT ON COLUMN consultation_diagnoses.line_number IS 'Order within the consultation, 1 = primary.';
COMMENT ON COLUMN consultation_diagnoses.condition_text IS 'Clinician-entered condition or diagnosis
text.';
COMMENT ON COLUMN consultation_diagnoses.icd10_code IS 'Optional ICD-10 code entered by the doctor
(format-checked only; no code catalogue in v1).';

-- ---------------------------------------------------------------------
-- medicines: lightweight medicine catalogue (no stock / inventory)
-- ---------------------------------------------------------------------
CREATE TABLE medicines (
    id          UUID         NOT NULL DEFAULT gen_random_uuid(),
    name        VARCHAR(200) NOT NULL,
    strength    VARCHAR(50),
```

```
    dosage_form  VARCHAR(20)  NOT NULL,
    active       BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    created_by   UUID         NOT NULL,
    updated_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    updated_by   UUID         NOT NULL,
    version      BIGINT       NOT NULL DEFAULT 0,
    CONSTRAINT pk_medicines PRIMARY KEY (id),
    CONSTRAINT ck_medicines_name_not_blank CHECK (btrim(name) <> ''),
    CONSTRAINT ck_medicines_dosage_form CHECK (dosage_form IN (
        'TABLET', 'CAPSULE', 'SYRUP', 'SUSPENSION', 'SOLUTION', 'INJECTION', 'CREAM', 'OINTMENT',
        'GEL', 'DROPS', 'INHALER', 'SUPPOSITORY', 'POWDER', 'PATCH', 'OTHER')),
    CONSTRAINT fk_medicines_created_by FOREIGN KEY (created_by) REFERENCES users (id),
    CONSTRAINT fk_medicines_updated_by FOREIGN KEY (updated_by) REFERENCES users (id)
);
COMMENT ON TABLE medicines IS 'Optional catalogue used for prescription autocomplete and to block
discontinued (inactive) medicines. Contains no stock levels: inventory is out of scope for v1.';
COMMENT ON COLUMN medicines.name IS 'Generic (or brand) name, e.g. "Paracetamol".';
COMMENT ON COLUMN medicines.strength IS 'Optional strength, e.g. "500 mg" or "125 mg/5 ml".';
COMMENT ON COLUMN medicines.dosage_form IS 'Dosage form enumeration.';
COMMENT ON COLUMN medicines.active IS 'FALSE = discontinued/unavailable; cannot be prescribed by
medicineId (422 MEDICINE_INACTIVE).';
CREATE UNIQUE INDEX uq_medicines_name_strength_form ON medicines (lower(name), lower(coalesce(strength,
'')), dosage_form);
CREATE INDEX ix_medicines_name_trgm ON medicines USING gin (name gin_trgm_ops);

-- ---------------------------------------------------------------------
-- prescriptions: medicine order produced by a consultation
-- ---------------------------------------------------------------------
CREATE SEQUENCE prescription_number_seq START WITH 1 INCREMENT BY 1 NO CYCLE;

CREATE TABLE prescriptions (
    id                       UUID          NOT NULL DEFAULT gen_random_uuid(),
    prescription_number      BIGINT        NOT NULL DEFAULT nextval('prescription_number_seq'),
    consultation_id          UUID          NOT NULL,
    patient_id               UUID          NOT NULL,
    status                   VARCHAR(12)   NOT NULL DEFAULT 'PENDING',
    notes                    VARCHAR(1000),
    submitted_at             TIMESTAMPTZ   NOT NULL DEFAULT now(),
    started_at               TIMESTAMPTZ,
    started_by               UUID,
    cancelled_at             TIMESTAMPTZ,
    cancelled_by             UUID,
    cancel_reason            VARCHAR(500),
    supersedes_prescription_id UUID,
    updated_at               TIMESTAMPTZ   NOT NULL DEFAULT now(),
    version                  BIGINT        NOT NULL DEFAULT 0,
    CONSTRAINT pk_prescriptions PRIMARY KEY (id),
    CONSTRAINT uq_prescriptions_number UNIQUE (prescription_number),
    CONSTRAINT uq_prescriptions_supersedes UNIQUE (supersedes_prescription_id),
    CONSTRAINT ck_prescriptions_status CHECK (status IN ('PENDING', 'IN_PROGRESS', 'DISPENSED', 'CANCELLED
')),
    CONSTRAINT ck_prescriptions_status_fields CHECK (
        (status <> 'IN_PROGRESS' OR started_at IS NOT NULL)
    AND (status <> 'CANCELLED' OR (cancelled_at IS NOT NULL AND cancelled_by IS NOT NULL AND
cancel_reason IS NOT NULL))
    AND (status = 'CANCELLED' OR cancelled_at IS NULL)),
    CONSTRAINT ck_prescriptions_not_self_superseding CHECK (supersedes_prescription_id IS NULL OR supersed
es_prescription_id <> id),
    CONSTRAINT fk_prescriptions_consultation_patient FOREIGN KEY (consultation_id, patient_id)
        REFERENCES consultations (id, patient_id),
    CONSTRAINT fk_prescriptions_patient FOREIGN KEY (patient_id) REFERENCES patients (id),
    CONSTRAINT fk_prescriptions_started_by FOREIGN KEY (started_by) REFERENCES users (id),
    CONSTRAINT fk_prescriptions_cancelled_by FOREIGN KEY (cancelled_by) REFERENCES users (id),
    CONSTRAINT fk_prescriptions_supersedes FOREIGN KEY (supersedes_prescription_id) REFERENCES prescriptio
ns (id)
);
COMMENT ON TABLE prescriptions IS 'Prescription header. Lifecycle PENDING -> IN_PROGRESS -> DISPENSED,
with CANCELLED from PENDING/IN_PROGRESS. DISPENSED and CANCELLED are terminal.';
COMMENT ON COLUMN prescriptions.prescription_number IS 'Human-readable sequential number, shown as
RX-000123 (format RX-%06d). Gaps are possible and harmless.';
COMMENT ON COLUMN prescriptions.consultation_id IS 'Consultation that produced the prescription. The
prescriber is consultations.doctor_id.';
COMMENT ON COLUMN prescriptions.patient_id IS 'Deliberate denormalisation for the pharmacist queue; kept
consistent with the consultation by a composite foreign key.';
COMMENT ON COLUMN prescriptions.status IS 'PENDING | IN_PROGRESS | DISPENSED | CANCELLED.';
COMMENT ON COLUMN prescriptions.notes IS 'Optional note from doctor to pharmacist.';
COMMENT ON COLUMN prescriptions.submitted_at IS 'Instant the prescription entered the pharmacist queue.';
COMMENT ON COLUMN prescriptions.started_at IS 'Instant a pharmacist marked it IN_PROGRESS.';
```

```
COMMENT ON COLUMN prescriptions.started_by IS 'Pharmacist who marked it IN_PROGRESS.';
COMMENT ON COLUMN prescriptions.cancelled_at IS 'Cancellation instant.';
COMMENT ON COLUMN prescriptions.cancelled_by IS 'Doctor (prescriber) or pharmacist who cancelled.';
COMMENT ON COLUMN prescriptions.cancel_reason IS 'Mandatory reason for cancellation.';
COMMENT ON COLUMN prescriptions.supersedes_prescription_id IS 'Cancelled prescription that this one
replaces (re-issue); each prescription can be superseded at most once.';
COMMENT ON COLUMN prescriptions.version IS 'Optimistic-locking version.';
CREATE UNIQUE INDEX uq_prescriptions_active_per_consultation ON prescriptions (consultation_id) WHERE stat
us <> 'CANCELLED';
CREATE INDEX ix_prescriptions_status_submitted ON prescriptions (status, submitted_at);
CREATE INDEX ix_prescriptions_patient_submitted ON prescriptions (patient_id, submitted_at DESC);
CREATE INDEX ix_prescriptions_consultation ON prescriptions (consultation_id);

-- ---------------------------------------------------------------------
-- prescription_items: what the doctor prescribed (immutable)
-- ---------------------------------------------------------------------
CREATE TABLE prescription_items (
    id              UUID          NOT NULL DEFAULT gen_random_uuid(),
    prescription_id  UUID          NOT NULL,
    line_number      SMALLINT      NOT NULL,
    medicine_id      UUID,
    medicine_name    VARCHAR(200)  NOT NULL,
    strength         VARCHAR(50),
    dosage_form      VARCHAR(20),
    dose             VARCHAR(100)  NOT NULL,
    frequency        VARCHAR(100)  NOT NULL,
    duration_days    SMALLINT,
    quantity         NUMERIC(7, 2) NOT NULL,
    quantity_unit    VARCHAR(30)   NOT NULL,
    instructions     VARCHAR(500),
    CONSTRAINT pk_prescription_items PRIMARY KEY (id),
    CONSTRAINT uq_prescription_items_line UNIQUE (prescription_id, line_number),
    CONSTRAINT uq_prescription_items_id_prescription UNIQUE (id, prescription_id),
    CONSTRAINT ck_prescription_items_line CHECK (line_number BETWEEN 1 AND 30),
    CONSTRAINT ck_prescription_items_name_not_blank CHECK (btrim(medicine_name) <> ''),
    CONSTRAINT ck_prescription_items_dosage_form CHECK (dosage_form IS NULL OR dosage_form IN (
        'TABLET', 'CAPSULE', 'SYRUP', 'SUSPENSION', 'SOLUTION', 'INJECTION', 'CREAM', 'OINTMENT',
        'GEL', 'DROPS', 'INHALER', 'SUPPOSITORY', 'POWDER', 'PATCH', 'OTHER')),
    CONSTRAINT ck_prescription_items_duration CHECK (duration_days IS NULL OR duration_days BETWEEN 1 AND
365),
    CONSTRAINT ck_prescription_items_quantity CHECK (quantity > 0),
    CONSTRAINT fk_prescription_items_prescription FOREIGN KEY (prescription_id) REFERENCES prescriptions
(id),
    CONSTRAINT fk_prescription_items_medicine FOREIGN KEY (medicine_id) REFERENCES medicines (id)
);
COMMENT ON TABLE prescription_items IS 'Prescribed lines (1-30 per prescription). medicine_name/strength/
dosage_form are snapshots so history is unchanged if the catalogue is edited later.';
COMMENT ON COLUMN prescription_items.line_number IS 'Order within the prescription.';
COMMENT ON COLUMN prescription_items.medicine_id IS 'Catalogue medicine, or NULL for a free-text
medicine.';
COMMENT ON COLUMN prescription_items.medicine_name IS 'Snapshot of the medicine name at prescribing
time.';
COMMENT ON COLUMN prescription_items.strength IS 'Snapshot of strength, e.g. "500 mg".';
COMMENT ON COLUMN prescription_items.dosage_form IS 'Snapshot of dosage form.';
COMMENT ON COLUMN prescription_items.dose IS 'Dose per administration, free text, e.g. "1 tablet", "5
ml".';
COMMENT ON COLUMN prescription_items.frequency IS 'Frequency, free text, e.g. "3 times daily (TDS)".';
COMMENT ON COLUMN prescription_items.duration_days IS 'Duration in days (1-365); NULL = as directed / as
needed.';
COMMENT ON COLUMN prescription_items.quantity IS 'Total quantity to dispense (BigDecimal, max 2
decimals).';
COMMENT ON COLUMN prescription_items.quantity_unit IS 'Unit of quantity, e.g. "tablets", "ml",
"bottles".';
COMMENT ON COLUMN prescription_items.instructions IS 'Optional patient instructions, e.g. "after meals".';

-- ---------------------------------------------------------------------
-- dispensings: the single dispensing event of a prescription
-- ---------------------------------------------------------------------
CREATE TABLE dispensings (
    id              UUID          NOT NULL DEFAULT gen_random_uuid(),
    prescription_id  UUID          NOT NULL,
    pharmacist_id    UUID          NOT NULL,
    dispensed_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
    outcome          VARCHAR(10)   NOT NULL,
    notes            VARCHAR(1000),
    CONSTRAINT pk_dispensings PRIMARY KEY (id),
    CONSTRAINT uq_dispensings_prescription UNIQUE (prescription_id),
    CONSTRAINT uq_dispensings_id_prescription UNIQUE (id, prescription_id),
    CONSTRAINT ck_dispensings_outcome CHECK (outcome IN ('FULL', 'PARTIAL')),
```

```
    CONSTRAINT fk_dispensings_prescription FOREIGN KEY (prescription_id) REFERENCES prescriptions (id),
    CONSTRAINT fk_dispensings_pharmacist FOREIGN KEY (pharmacist_id) REFERENCES users (id)
);
COMMENT ON TABLE dispensings IS 'Who dispensed, when, and the overall outcome. UNIQUE(prescription_id)
makes duplicate dispensing impossible at the database level.';
COMMENT ON COLUMN dispensings.pharmacist_id IS 'Authenticated pharmacist who confirmed dispensing.';
COMMENT ON COLUMN dispensings.dispensed_at IS 'Server instant of confirmation.';
COMMENT ON COLUMN dispensings.outcome IS 'FULL (every item fully dispensed) | PARTIAL (at least one item
partial or not dispensed). Derived by the server.';
COMMENT ON COLUMN dispensings.notes IS 'Optional pharmacist note.';

-- ---------------------------------------------------------------------
-- dispensing_items: what was actually handed over, per prescribed line
-- ---------------------------------------------------------------------
CREATE TABLE dispensing_items (
    id                  UUID          NOT NULL DEFAULT gen_random_uuid(),
    dispensing_id       UUID          NOT NULL,
    prescription_id     UUID          NOT NULL,
    prescription_item_id  UUID        NOT NULL,
    quantity_dispensed  NUMERIC(7, 2) NOT NULL,
    status              VARCHAR(15)   NOT NULL,
    note                VARCHAR(500),
    CONSTRAINT pk_dispensing_items PRIMARY KEY (id),
    CONSTRAINT uq_dispensing_items_item UNIQUE (prescription_item_id),
    CONSTRAINT ck_dispensing_items_quantity CHECK (quantity_dispensed >= 0),
    CONSTRAINT ck_dispensing_items_status CHECK (status IN ('FULL', 'PARTIAL', 'NOT_DISPENSED')),
    CONSTRAINT ck_dispensing_items_zero_iff_not_dispensed CHECK ((status = 'NOT_DISPENSED') = (quantity_di
spensed = 0)),
    CONSTRAINT ck_dispensing_items_note_required CHECK (status = 'FULL' OR note IS NOT NULL),
    CONSTRAINT fk_dispensing_items_dispensing FOREIGN KEY (dispensing_id, prescription_id)
        REFERENCES dispensings (id, prescription_id),
    CONSTRAINT fk_dispensing_items_item FOREIGN KEY (prescription_item_id, prescription_id)
        REFERENCES prescription_items (id, prescription_id)
);
COMMENT ON TABLE dispensing_items IS 'One row per prescription item in the dispensing. Composite foreign
keys guarantee the item belongs to the same prescription as the dispensing.';
COMMENT ON COLUMN dispensing_items.prescription_id IS 'Redundant copy used only by the composite foreign
keys.';
COMMENT ON COLUMN dispensing_items.quantity_dispensed IS 'Quantity handed over (0 .. prescribed quantity,
max 2 decimals).';
COMMENT ON COLUMN dispensing_items.status IS 'FULL | PARTIAL | NOT_DISPENSED, derived by the server from
quantity_dispensed vs prescribed quantity.';
COMMENT ON COLUMN dispensing_items.note IS 'Mandatory when not FULL, e.g. "out of stock", "brand
substituted".';
CREATE INDEX ix_dispensing_items_dispensing ON dispensing_items (dispensing_id);
CREATE INDEX ix_dispensings_pharmacist_at ON dispensings (pharmacist_id, dispensed_at DESC);

-- ---------------------------------------------------------------------
-- idempotency_keys: de-duplication of retried create requests
-- ---------------------------------------------------------------------
CREATE TABLE idempotency_keys (
    id              UUID         NOT NULL DEFAULT gen_random_uuid(),
    user_id         UUID         NOT NULL,
    idempotency_key  VARCHAR(100) NOT NULL,
    operation       VARCHAR(50)  NOT NULL,
    request_hash    CHAR(64)     NOT NULL,
    resource_id     UUID,
    response_status  SMALLINT,
    created_at      TIMESTAMPTZ  NOT NULL DEFAULT now(),
    expires_at      TIMESTAMPTZ  NOT NULL,
    CONSTRAINT pk_idempotency_keys PRIMARY KEY (id),
    CONSTRAINT uq_idempotency_keys_user_key UNIQUE (user_id, idempotency_key),
    CONSTRAINT ck_idempotency_keys_key_format CHECK (idempotency_key ~ '^[A-Za-z0-9_-]{8,100}$'),
    CONSTRAINT ck_idempotency_keys_hash_hex CHECK (request_hash ~ '^[0-9a-f]{64}$'),
    CONSTRAINT fk_idempotency_keys_user FOREIGN KEY (user_id) REFERENCES users (id)
);
COMMENT ON TABLE idempotency_keys IS
'Inserted in the SAME transaction as the business write. A concurrent duplicate blocks on the unique index
until the first commits, then replays the stored resource. Stores ids only, never response bodies (no PHI
copies).';
COMMENT ON COLUMN idempotency_keys.idempotency_key IS 'Client-supplied Idempotency-Key header (UUID
recommended).';
COMMENT ON COLUMN idempotency_keys.operation IS 'Operation id, e.g. createConsultation. Same key with a
different operation or body -> 422 IDEMPOTENCY_KEY_REUSED.';
COMMENT ON COLUMN idempotency_keys.request_hash IS 'SHA-256 of operation + path + canonical JSON body.';
COMMENT ON COLUMN idempotency_keys.resource_id IS 'Id of the created resource, used to rebuild the
response on replay.';
COMMENT ON COLUMN idempotency_keys.response_status IS 'HTTP status of the original response (201 or
200).';
```

```sql
COMMENT ON COLUMN idempotency_keys.expires_at IS 'created_at + 24 h; purged by the retention job.';
CREATE INDEX ix_idempotency_keys_expires_at ON idempotency_keys (expires_at);

-- ---------------------------------------------------------------------
-- audit_events: append-only security and clinical audit trail
-- ---------------------------------------------------------------------
CREATE TABLE audit_events (
    id              UUID         NOT NULL DEFAULT gen_random_uuid(),
    occurred_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),
    actor_user_id   UUID,
    actor_username  VARCHAR(50),
    action          VARCHAR(50)  NOT NULL,
    entity_type     VARCHAR(30)  NOT NULL,
    entity_id       UUID,
    patient_id      UUID,
    outcome         VARCHAR(10)  NOT NULL DEFAULT 'SUCCESS',
    details         JSONB        NOT NULL DEFAULT '{}'::jsonb,
    request_id      VARCHAR(64),
    ip_address      INET,
    user_agent      VARCHAR(255),
    CONSTRAINT pk_audit_events PRIMARY KEY (id),
    CONSTRAINT ck_audit_events_action_format CHECK (action ~ '^[A-Z][A-Z_]{2,49}$'),
    CONSTRAINT ck_audit_events_entity_type CHECK (entity_type IN (
        'USER', 'SESSION', 'PATIENT', 'CONSULTATION', 'PRESCRIPTION', 'MEDICINE',
        'QUEUE_SESSION', 'QUEUE_ENTRY', 'AUDIT')),
    CONSTRAINT ck_audit_events_outcome CHECK (outcome IN ('SUCCESS', 'FAILURE', 'DENIED')),
    CONSTRAINT ck_audit_events_details_object CHECK (jsonb_typeof(details) = 'object'),
    CONSTRAINT fk_audit_events_actor FOREIGN KEY (actor_user_id) REFERENCES users (id),
    CONSTRAINT fk_audit_events_patient FOREIGN KEY (patient_id) REFERENCES patients (id)
);
COMMENT ON TABLE audit_events IS
'Append-only audit trail (UPDATE/DELETE blocked by trigger). details must never contain passwords, tokens,
clinical notes or diagnoses.';
COMMENT ON COLUMN audit_events.occurred_at IS 'Event instant (UTC).';
COMMENT ON COLUMN audit_events.actor_user_id IS 'Authenticated user, NULL for anonymous events (e.g.
failed login of unknown user).';
COMMENT ON COLUMN audit_events.actor_username IS 'Username snapshot (only for existing users).';
COMMENT ON COLUMN audit_events.action IS 'AuditAction enumeration value, e.g. PATIENT_HISTORY_VIEWED.';
COMMENT ON COLUMN audit_events.entity_type IS 'Type of the affected entity.';
COMMENT ON COLUMN audit_events.entity_id IS 'Id of the affected entity.';
COMMENT ON COLUMN audit_events.patient_id IS 'Patient whose data was touched; supports "who accessed this
patient" reports.';
COMMENT ON COLUMN audit_events.outcome IS 'SUCCESS | FAILURE | DENIED.';
COMMENT ON COLUMN audit_events.details IS 'Small JSON object with non-sensitive context (e.g.
{"fromStatus":"PENDING","toStatus":"DISPENSED"}).';
COMMENT ON COLUMN audit_events.request_id IS 'X-Request-Id correlating with application logs.';
COMMENT ON COLUMN audit_events.ip_address IS 'Client IP (from the trusted proxy).';
COMMENT ON COLUMN audit_events.user_agent IS 'Truncated User-Agent.';
CREATE INDEX ix_audit_events_occurred_at ON audit_events (occurred_at DESC);
CREATE INDEX ix_audit_events_patient ON audit_events (patient_id, occurred_at DESC) WHERE patient_id IS NO
T NULL;
CREATE INDEX ix_audit_events_actor ON audit_events (actor_user_id, occurred_at DESC);
CREATE INDEX ix_audit_events_entity ON audit_events (entity_type, entity_id);

CREATE FUNCTION audit_events_block_modification() RETURNS trigger
    LANGUAGE plpgsql AS
$$
BEGIN
    RAISE EXCEPTION 'audit_events is append-only (% blocked)', TG_OP
        USING ERRCODE = 'insufficient_privilege';
END;
$$;

CREATE TRIGGER trg_audit_events_append_only
    BEFORE UPDATE OR DELETE ON audit_events
    FOR EACH ROW EXECUTE FUNCTION audit_events_block_modification();
```

6. Example queries

Each query was executed against the development seed data. Spring Data implementations must produce equivalent SQL (verified with `spring.jpa.show-sql` in tests).

Q1 Look up a patient by patient number (exact, normalised to upper case)

Used by GET /api/v1/patients/by-number/{patientNumber}. Index: uq_patients_patient_number.

```
SELECT id, patient_number, full_name, sex, date_of_birth, date_of_birth_estimated,
       date_part('year', age(current_date, date_of_birth))::int AS age_years
FROM patients
WHERE patient_number = upper(btrim('p-0001'));
```

Q2 Search patients by name fragment, phone or patient-number prefix

Used by GET /api/v1/patients?search=. Index: ix_patients_full_name_trgm (ILIKE), uq_patients_patient_number (prefix).

```
SELECT id, patient_number, full_name, sex, date_of_birth
FROM patients
WHERE full_name ILIKE '%' || 'kumara' || '%'
   OR patient_number LIKE upper('kumara') || '%'
   OR phone_number = regexp_replace('kumara', '[\s()-]', '', 'g')
ORDER BY full_name, patient_number
LIMIT 20 OFFSET 0;
```

Q3 Medical history, step 1: one page of consultations, newest first

Used by GET /api/v1/patients/{patientId}/medical-history. Index: ix_consultations_patient_at.

```
SELECT c.id, c.consultation_at, c.presenting_complaint, c.clinical_notes, u.full_name AS doctor_name
FROM consultations c
JOIN users u ON u.id = c.doctor_id
WHERE c.patient_id = '20000000-0000-4000-8000-000000000001'
ORDER BY c.consultation_at DESC, c.id
LIMIT 10 OFFSET 0;
```

Q4 Medical history, step 2: batch-load prescriptions, items and dispensing for the page (no N+1)

The id list comes from Q3. Diagnoses are batch-loaded the same way (WHERE consultation_id = ANY(:ids)).

```
SELECT p.consultation_id, p.id AS prescription_id, p.prescription_number, p.status,
       pi.line_number, pi.medicine_name, pi.strength, pi.dose, pi.frequency, pi.quantity,
pi.quantity_unit,
       d.outcome AS dispensing_outcome, d.dispensed_at, di.quantity_dispensed, di.status AS item_dispense_
status
FROM prescriptions p
JOIN prescription_items pi ON pi.prescription_id = p.id
LEFT JOIN dispensings d ON d.prescription_id = p.id
LEFT JOIN dispensing_items di ON di.prescription_item_id = pi.id
WHERE p.consultation_id = ANY (ARRAY['40000000-0000-4000-8000-000000000001',
                                     '40000000-0000-4000-8000-000000000002']::uuid[])
ORDER BY p.consultation_id, p.submitted_at DESC, pi.line_number;
```

Q5 Previous medical conditions summary for a patient

Used for MedicalHistoryResponse.conditions.

```
SELECT min(cd.condition_text)              AS condition_text,
       max(cd.icd10_code)                 AS icd10_code,
       max(c.consultation_at)             AS last_recorded_at,
       count(*)                           AS occurrences
FROM consultation_diagnoses cd
JOIN consultations c ON c.id = cd.consultation_id
WHERE c.patient_id = '20000000-0000-4000-8000-000000000001'
```

```
GROUP BY lower(btrim(cd.condition_text))
ORDER BY last_recorded_at DESC
LIMIT 50;
```

Q6 Pharmacist queue: active prescriptions, oldest first

Used by GET /api/v1/prescriptions?status=PENDING&status=IN_PROGRESS. Index: ix_prescriptions_status_submitted.

```
SELECT p.id, p.prescription_number, p.consultation_id, p.status, p.submitted_at,
       pt.patient_number, pt.full_name AS patient_name, u.full_name AS prescribed_by,
       (SELECT count(*) FROM prescription_items pi WHERE pi.prescription_id = p.id) AS item_count
FROM prescriptions p
JOIN patients pt ON pt.id = p.patient_id
JOIN consultations c ON c.id = p.consultation_id
JOIN users u ON u.id = c.doctor_id
WHERE p.status IN ('PENDING', 'IN_PROGRESS')
ORDER BY p.submitted_at ASC, p.id
LIMIT 20 OFFSET 0;
```

Q7 Issue the next queue number (inside the booking transaction)

The UPDATE takes a row lock on the session; concurrent bookings queue behind it. If the transaction rolls back, the increment rolls back too, so numbers stay gap-free.

```
BEGIN;
SELECT id, status, last_issued_number FROM queue_sessions
WHERE id = '30000000-0000-4000-8000-000000000001' FOR UPDATE;
UPDATE queue_sessions
   SET last_issued_number = last_issued_number + 1, updated_at = now(), version = version + 1
 WHERE id = '30000000-0000-4000-8000-000000000001'
   AND status IN ('SCHEDULED', 'OPEN')
   AND last_issued_number < 999
RETURNING last_issued_number;
ROLLBACK;
```

Q8 Call next: pick the lowest waiting number (inside the call-next transaction)

Preceded by SELECT ... FOR UPDATE on the session row, so no SKIP LOCKED is needed. Index: ix_queue_entries_session_status_number.

```
SELECT id, queue_number
FROM queue_entries
WHERE queue_session_id = '30000000-0000-4000-8000-000000000001'
  AND status = 'WAITING'
ORDER BY queue_number
LIMIT 1;
```

Q9 Public queue status (no personal data selected)

Used by GET /api/v1/public/queue-sessions/{id}.

```
SELECT s.id, s.name, s.session_date, s.status, s.last_called_number AS current_serving_number,
       s.last_issued_number, s.version, s.updated_at,
       (SELECT coalesce(array_agg(e.queue_number ORDER BY e.queue_number), '{}')
          FROM queue_entries e WHERE e.queue_session_id = s.id AND e.status = 'SERVING') AS serving_number
s,
       (SELECT count(*) FROM queue_entries e WHERE e.queue_session_id = s.id AND e.status = 'WAITING') AS
waiting_count,
       (SELECT coalesce(array_agg(n ORDER BY n), '{}') FROM (
            SELECT e.queue_number AS n FROM queue_entries e
             WHERE e.queue_session_id = s.id AND e.status = 'WAITING'
             ORDER BY e.queue_number LIMIT 50) w) AS waiting_numbers
FROM queue_sessions s
WHERE s.id = '30000000-0000-4000-8000-000000000001';
```

Q10 Who accessed a patient's record (audit report)

Used by GET /api/v1/audit-events?patientId=. Index: ix_audit_events_patient.

```
SELECT a.occurred_at, a.actor_username, a.action, a.outcome, a.ip_address
FROM audit_events a
WHERE a.patient_id = '20000000-0000-4000-8000-000000000001'
ORDER BY a.occurred_at DESC
LIMIT 50;
```

Q11 Prescriptions dispensed by each pharmacist today (dispensary time zone)

```
SELECT u.full_name AS pharmacist, d.outcome, count(*) AS dispensed
FROM dispensings d
JOIN users u ON u.id = d.pharmacist_id
WHERE (d.dispensed_at AT TIME ZONE 'Asia/Colombo')::date = (now() AT TIME ZONE 'Asia/Colombo')::date
GROUP BY u.full_name, d.outcome
ORDER BY u.full_name, d.outcome;
```

Q12 Retention job: anonymise queue entries older than 90 days

```
UPDATE queue_entries
   SET patient_name = NULL, contact_phone = NULL, notes = NULL,
       anonymized_at = now(), updated_at = now(), version = version + 1
 WHERE anonymized_at IS NULL
   AND booked_at < now() - INTERVAL '90 days'
   AND status IN ('COMPLETED', 'SKIPPED', 'CANCELLED');
```

7. Transactions and concurrency

7.1 Principles

- One application command = one database transaction, opened by the service method (`@Transactional`); controllers never open transactions.
- Isolation level READ COMMITTED (PostgreSQL default) plus explicit row locks where a decision depends on current state.
- Every connection sets `lock_timeout = '5s'` and `statement_timeout = '15s'` (Hikari `connection-init-sql`). A lock timeout (SQLSTATE `55P03`) becomes `503 SERVICE_UNAVAILABLE` (retryable with the same idempotency key).
- **Fixed lock order** to avoid deadlocks: queue operations lock the `queue_sessions` row first, then entries; pharmacy operations lock the `prescriptions` row only; the last-admin check locks admin `users` rows ordered by id.
- Invariants are enforced twice: by the service (friendly error) and by a database constraint (final guard). Constraint names are mapped to error codes.

7.2 Transaction boundaries and locking per operation

Operation	Transaction contents	Concurrency control	Failure behaviour
createPatient	idempotency row, patient, audit	`uq_patients_patient_number`; idempotency unique key	409 DUPLICATE_PATIENT_NUMBER, replay on same key
updatePatient	patient update, audit	`@Version` optimistic lock	409 CONCURRENT_MODIFICATION
createConsultation	idempotency row, consultation, diagnoses, prescription, items, audit	idempotency unique key; `uq_consultations_queue_entry`; FK checks	any error rolls back all rows
addPrescriptionToConsultation	idempotency row, prescription, items, audit	`uq_prescriptions_active_per_consultation`, `uq_prescriptions_supersedes`	409 ACTIVE_PRESCRIPTION_EXISTS
startPrescription	status update, audit	`SELECT ... FOR UPDATE` on prescription	409 on terminal states
dispensePrescription	idempotency row, lock, dispensing, items, status, audit	`FOR UPDATE` + `uq_dispensings_prescription` + idempotency	409 INVALID_STATUS_TRANSITION for the loser
cancelPrescription	lock, status, audit	`FOR UPDATE`	409 when already dispensed
createBooking	idempotency row, lock session, counter + 1, entry, audit	`FOR UPDATE` on session; `uq_queue_entries_session_number`	rollback undoes the counter (no gaps)
callNextPatient	lock session, compare expected number, complete current, pick lowest WAITING, update entry and session, audit	`FOR UPDATE` on session + compare-and-set	409 QUEUE_STATE_CHANGED, nothing changed
call/complete/skip/requeue/cancel entry	lock session, entry transition, session version + 1, audit	`FOR UPDATE` on session	409 INVALID_STATUS_TRANSITION
open/close session	lock session, bulk entry updates (force), audit	`FOR UPDATE` on session	409 QUEUE_SESSION_HAS_ACTIVE_ENTRIES
refreshToken	lock token row, rotate, insert successor	`FOR UPDATE` on refresh token	401 REFRESH_TOKEN_ROTATED for the parallel loser
updateUser	users + roles, revoke tokens, audit	`@Version`; `FOR UPDATE` on enabled admins for last-admin check	409 / 422 LAST_ADMIN_REQUIRED
read endpoints	read-only transaction (+ audit insert for audited reads)	none	-

7.3 Queue number generation (gap-free)

```
-- inside the booking transaction (QueueEntryService.createBooking)
SELECT * FROM queue_sessions WHERE id = :sid FOR UPDATE;          -- serialises bookings per session
```

```
-- check status IN ('SCHEDULED','OPEN'), last_issued_number < 999, duplicate phone
UPDATE queue_sessions SET last_issued_number = last_issued_number + 1,
       version = version + 1, updated_at = now() WHERE id = :sid;   -- n + 1
INSERT INTO queue_entries (..., queue_number, status) VALUES (..., :n_plus_1, 'WAITING');
COMMIT;                                                              -- or ROLLBACK: counter restored
```

A PostgreSQL sequence was rejected: sequences are not transactional (a rolled-back booking would leave a gap) and one sequence per session would be needed. The row lock is held for milliseconds; at 300 bookings per day contention is negligible.

7.4 Call next (compare-and-set)

The request carries `expectedLastCalledNumber`. Under the session row lock the service compares it with `last_called_number` (both may be null). Equal → optional completion of `completeCurrentEntryId`, selection of the lowest WAITING entry, update to SERVING, `last_called_number := number`, `version + 1`. Not equal → `409 QUEUE_STATE_CHANGED` and no change. Two concurrent presses based on the same screen therefore advance the queue once; a network retry of a successful call fails the compare and simply refreshes the screen.

7.5 Idempotency mechanism

1. The service computes `request_hash = SHA-256(operationId + path + canonical JSON body)`.
2. In the business transaction it executes `INSERT INTO idempotency_keys (...) ON CONFLICT (user_id, idempotency_key) DO NOTHING RETURNING id`. If a concurrent request with the same key is still running, PostgreSQL blocks this insert on the unique index until that transaction ends.
3. Inserted → perform the operation, then store `resource_id` and `response_status` in the same transaction.
4. Not inserted → read the existing row: different `operation` or `request_hash` → `422 IDEMPOTENCY_KEY_REUSED`; otherwise rebuild the response from `resource_id` (current state of the resource) with the original status (replay). No response bodies (and therefore no PHI) are stored.
5. Keys expire after 24 h and are purged by the retention job.

7.6 Verification performed for this document

The following checks were run against PostgreSQL 16 while preparing this document; the backend test suite repeats them (BT-013, BT-017, BT-019):

- Duplicate dispensing, foreign dispensing item, second active prescription, prescription/patient mismatch, duplicate patient number, lower-case patient number, duplicate queue number, invalid status, cancellation without reason, inconsistent NOT_DISPENSED quantity, duplicate session name (case-insensitive), serving number beyond issued number, OPEN without opened_at, UPDATE and DELETE on `audit_events` - **all rejected by the database**.
- 60 concurrent booking transactions (every 10th rolled back) produced exactly numbers 1..54 with no gaps or duplicates and `last_issued_number = 54`.

8. Flyway migration strategy

- Location `classpath:db/migration` (all environments) and `classpath:db/seed` (only profile `dev`, repeatable `R__dev_seed.sql`).
- Naming `V<n>__<snake_description>.sql`, sequential integers; applied migrations are **never edited** (checksum validation stays on, `spring.flyway.validate-on-migrate=true`, `clean-disabled=true`).
- Hibernate never changes the schema: `spring.jpa.hibernate.ddl-auto=validate` in all profiles (BT-025 fails on drift).
- Every schema change ships as a new migration in the same pull request as the entity change; destructive changes use expand/contract (add column → backfill → switch code → drop in a later release).
- New enum values: a migration that drops and recreates the CHECK constraint, plus the Java enum and `openapi.yaml` in the same change.
- Migrations run at application start (Flyway takes a PostgreSQL advisory lock, so concurrent starts are safe); with several replicas, start one replica first and roll out the others after it is healthy.
- Production database credentials for migrations can be separated with `DB_MIGRATION_USERNAME` / `DB_MIGRATION_PASSWORD`.

9. Development seed data

`R__dev_seed.sql` is idempotent (fixed UUIDs, `ON CONFLICT DO NOTHING`) and loaded only in the `dev` profile. It provides one user per role (plus a doctor-pharmacist), 11 medicines (one inactive), 5 patients (one with an estimated DOB), an OPEN session for today with entries in several states, a SCHEDULED session for tomorrow, one partially dispensed and one pending prescription. All dev users have the password `DevPassword#2026` and are not forced to change it. **Never load this file in production.**

```
-- ================================================================
-- DEVELOPMENT SEED DATA ONLY - never deploy to production.
-- Flyway repeatable migration, location classpath:db/seed, which is
-- added to spring.flyway.locations ONLY in the "dev" profile.
-- Idempotent: fixed UUIDs + ON CONFLICT DO NOTHING.
-- All dev users share the password  DevPassword#2026
-- ================================================================

-- ---------- users ----------
INSERT INTO users (id, username, password_hash, full_name, professional_registration_number,
                   enabled, must_change_password, created_at, updated_at)
VALUES
 ('00000000-0000-4000-8000-000000000001', 'admin',
  '{bcrypt}$2b$12$VpsfzT1SzLJVRTehnxHfF.lFUgiUWaruMhfxLw2F3koKxXJ4jkeCC',
  'System Administrator', NULL, TRUE, FALSE, now(), now()),
 ('00000000-0000-4000-8000-000000000002', 'dr.perera',
  '{bcrypt}$2b$12$VpsfzT1SzLJVRTehnxHfF.lFUgiUWaruMhfxLw2F3koKxXJ4jkeCC',
  'Dr. Nimal Perera', 'MC-12345', TRUE, FALSE, now(), now()),
 ('00000000-0000-4000-8000-000000000003', 'dr.fernando',
  '{bcrypt}$2b$12$VpsfzT1SzLJVRTehnxHfF.lFUgiUWaruMhfxLw2F3koKxXJ4jkeCC',
  'Dr. Anusha Fernando', 'MC-23456', TRUE, FALSE, now(), now()),
 ('00000000-0000-4000-8000-000000000004', 'ph.silva',
  '{bcrypt}$2b$12$VpsfzT1SzLJVRTehnxHfF.lFUgiUWaruMhfxLw2F3koKxXJ4jkeCC',
  'Kamal Silva', 'PH-3456', TRUE, FALSE, now(), now()),
 ('00000000-0000-4000-8000-000000000005', 'dr.jayasinghe',
  '{bcrypt}$2b$12$VpsfzT1SzLJVRTehnxHfF.lFUgiUWaruMhfxLw2F3koKxXJ4jkeCC',
  'Dr. Ruwan Jayasinghe (doctor + pharmacist)', 'MC-34567', TRUE, FALSE, now(), now())
ON CONFLICT (id) DO NOTHING;

INSERT INTO user_roles (user_id, role) VALUES
 ('00000000-0000-4000-8000-000000000001', 'ADMIN'),
 ('00000000-0000-4000-8000-000000000002', 'DOCTOR'),
 ('00000000-0000-4000-8000-000000000003', 'DOCTOR'),
 ('00000000-0000-4000-8000-000000000004', 'PHARMACIST'),
 ('00000000-0000-4000-8000-000000000005', 'DOCTOR'),
 ('00000000-0000-4000-8000-000000000005', 'PHARMACIST')
ON CONFLICT DO NOTHING;

-- ---------- medicines ----------
INSERT INTO medicines (id, name, strength, dosage_form, active, created_by, updated_by) VALUES
 ('10000000-0000-4000-8000-000000000001', 'Paracetamol',  '500 mg',       'TABLET',  TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000002', 'Paracetamol',  '120 mg/5 ml',  'SYRUP',   TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000003', 'Amoxicillin',  '500 mg',       'CAPSULE', TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000004', 'Amoxicillin',  '125 mg/5 ml',  'SUSPENSION', TRUE, '00000000-00
00-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000005', 'Metformin',    '500 mg',       'TABLET',  TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000006', 'Amlodipine',   '5 mg',         'TABLET',  TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000007', 'Cetirizine',   '10 mg',        'TABLET',  TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000008', 'Omeprazole',   '20 mg',        'CAPSULE', TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000009', 'Salbutamol',   '100 mcg/dose', 'INHALER', TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000010', 'Oral rehydration salts', NULL, 'POWDER',  TRUE,  '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004'),
 ('10000000-0000-4000-8000-000000000011', 'Chlorphenamine', '4 mg',       'TABLET',  FALSE, '00000000-0000
-4000-8000-000000000004', '00000000-0000-4000-8000-000000000004')
ON CONFLICT (id) DO NOTHING;

-- ---------- patients ----------
```

```
INSERT INTO patients (id, patient_number, full_name, sex, date_of_birth, date_of_birth_estimated,
                      phone_number, address, known_allergies, created_by, updated_by) VALUES
 ('20000000-0000-4000-8000-000000000001', 'P-0001', 'Saman Kumara',     'MALE',   DATE '1978-04-12',
FALSE, '+94771234567', '12 Temple Road, Kandy', 'Penicillin (rash)',
  '00000000-0000-4000-8000-000000000002', '00000000-0000-4000-8000-000000000002'),
 ('20000000-0000-4000-8000-000000000002', 'P-0002', 'Dilani Wijesekara', 'FEMALE', DATE '1990-11-03', FALS
E, '0712345678',   NULL, NULL,
  '00000000-0000-4000-8000-000000000002', '00000000-0000-4000-8000-000000000002'),
 ('20000000-0000-4000-8000-000000000003', 'P-0003', 'Ishara Gunasekara', 'FEMALE', DATE '2019-06-20', FALS
E, '0759876543',   NULL, NULL,
  '00000000-0000-4000-8000-000000000003', '00000000-0000-4000-8000-000000000003'),
 ('20000000-0000-4000-8000-000000000004', 'P-0004', 'Piyadasa Herath',   'MALE',   (current_date - INTERVA
L '70 years')::date, TRUE, NULL, 'Matale', NULL,
  '00000000-0000-4000-8000-000000000003', '00000000-0000-4000-8000-000000000003'),
 ('20000000-0000-4000-8000-000000000005', 'CARD-77812', 'Fathima Rizvi',  'FEMALE', DATE '1985-02-28', FAL
SE, NULL, NULL, 'None known',
  '00000000-0000-4000-8000-000000000002', '00000000-0000-4000-8000-000000000002')
ON CONFLICT (id) DO NOTHING;

-- ---------- queue session for today (OPEN) ----------
INSERT INTO queue_sessions (id, session_date, name, assigned_doctor_id, status,
                            last_issued_number, last_called_number, opened_at, opened_by, created_by)
VALUES ('30000000-0000-4000-8000-000000000001', current_date, 'Morning OPD - Room 1',
        '00000000-0000-4000-8000-000000000002', 'OPEN', 5, 2, now() - INTERVAL '2 hours',
        '00000000-0000-4000-8000-000000000002', '00000000-0000-4000-8000-000000000004')
ON CONFLICT (id) DO NOTHING;

INSERT INTO queue_sessions (id, session_date, name, assigned_doctor_id, status, created_by)
VALUES ('30000000-0000-4000-8000-000000000002', current_date + 1, 'Morning OPD - Room 1',
        '00000000-0000-4000-8000-000000000002', 'SCHEDULED', '00000000-0000-4000-8000-000000000004')
ON CONFLICT (id) DO NOTHING;

INSERT INTO queue_entries (id, queue_session_id, queue_number, status, channel, patient_name, contact_phon
e,
                           booked_at, booked_by, called_at, called_by, completed_at, completed_by)
VALUES
 ('31000000-0000-4000-8000-000000000001', '30000000-0000-4000-8000-000000000001', 1, 'COMPLETED', 'PHONE',
  'Saman Kumara', '+94771234567', now() - INTERVAL '3 hours', '00000000-0000-4000-8000-000000000004',
  now() - INTERVAL '100 minutes', '00000000-0000-4000-8000-000000000002',
  now() - INTERVAL '90 minutes', '00000000-0000-4000-8000-000000000002'),
 ('31000000-0000-4000-8000-000000000002', '30000000-0000-4000-8000-000000000001', 2, 'SERVING', 'WALK_IN',
  'Dilani Wijesekara', NULL, now() - INTERVAL '150 minutes', '00000000-0000-4000-8000-000000000004',
  now() - INTERVAL '10 minutes', '00000000-0000-4000-8000-000000000002', NULL, NULL),
 ('31000000-0000-4000-8000-000000000003', '30000000-0000-4000-8000-000000000001', 3, 'WAITING', 'PHONE',
  'Ishara Gunasekara (mother calling)', '0759876543', now() - INTERVAL '140 minutes', '00000000-0000-4000-
8000-000000000004',
  NULL, NULL, NULL, NULL),
 ('31000000-0000-4000-8000-000000000004', '30000000-0000-4000-8000-000000000001', 4, 'WAITING', 'PHONE',
  'Mr. Bandara', '0723456789', now() - INTERVAL '120 minutes', '00000000-0000-4000-8000-000000000004',
  NULL, NULL, NULL, NULL),
 ('31000000-0000-4000-8000-000000000005', '30000000-0000-4000-8000-000000000001', 5, 'WAITING', 'WALK_IN',
  'Ms. Rizvi', NULL, now() - INTERVAL '30 minutes', '00000000-0000-4000-8000-000000000004',
  NULL, NULL, NULL, NULL)
ON CONFLICT (id) DO NOTHING;

-- ---------- consultation 1 (yesterday, dispensed, partial) ----------
INSERT INTO consultations (id, patient_id, doctor_id, queue_entry_id, consultation_at, presenting_complain
t, clinical_notes)
VALUES ('40000000-0000-4000-8000-000000000001', '20000000-0000-4000-8000-000000000001',
        '00000000-0000-4000-8000-000000000002', NULL, now() - INTERVAL '30 days',
        'Fever and sore throat for 3 days', 'Temp 38.4 C. Tonsils inflamed, no exudate. Advised fluids.')
ON CONFLICT (id) DO NOTHING;
INSERT INTO consultation_diagnoses (id, consultation_id, line_number, condition_text, icd10_code) VALUES
 ('41000000-0000-4000-8000-000000000001', '40000000-0000-4000-8000-000000000001', 1, 'Acute pharyngitis',
'J02.9')
ON CONFLICT (id) DO NOTHING;
INSERT INTO prescriptions (id, prescription_number, consultation_id, patient_id, status, notes, submitted_
at, started_at, started_by, updated_at)
VALUES ('50000000-0000-4000-8000-000000000001', 1, '40000000-0000-4000-8000-000000000001',
        '20000000-0000-4000-8000-000000000001', 'DISPENSED', NULL, now() - INTERVAL '30 days',
        now() - INTERVAL '30 days' + INTERVAL '5 minutes', '00000000-0000-4000-8000-000000000004', now()
- INTERVAL '30 days')
ON CONFLICT (id) DO NOTHING;
INSERT INTO prescription_items (id, prescription_id, line_number, medicine_id, medicine_name, strength, do
sage_form,
                                dose, frequency, duration_days, quantity, quantity_unit, instructions) VAL
UES
 ('51000000-0000-4000-8000-000000000001', '50000000-0000-4000-8000-000000000001', 1, '10000000-0000-4000-8
000-000000000001',
```

```
  'Paracetamol', '500 mg', 'TABLET', '2 tablets', '4 times daily (QDS) when needed', 3, 24, 'tablets', 'Ma
ximum 8 tablets in 24 hours'),
 ('51000000-0000-4000-8000-000000000002', '50000000-0000-4000-8000-000000000001', 2, NULL,
  'Chlorhexidine mouthwash', '0.2%', 'SOLUTION', '10 ml', 'Twice daily (BD)', 5, 1, 'bottle', 'Rinse and
spit, do not swallow')
ON CONFLICT (id) DO NOTHING;
INSERT INTO dispensings (id, prescription_id, pharmacist_id, dispensed_at, outcome, notes)
VALUES ('52000000-0000-4000-8000-000000000001', '50000000-0000-4000-8000-000000000001',
        '00000000-0000-4000-8000-000000000004', now() - INTERVAL '30 days' + INTERVAL '12 minutes', 'PARTI
AL', NULL)
ON CONFLICT (id) DO NOTHING;
INSERT INTO dispensing_items (id, dispensing_id, prescription_id, prescription_item_id,
quantity_dispensed, status, note) VALUES
 ('53000000-0000-4000-8000-000000000001', '52000000-0000-4000-8000-000000000001', '50000000-0000-4000-8000
-000000000001',
  '51000000-0000-4000-8000-000000000001', 24, 'FULL', NULL),
 ('53000000-0000-4000-8000-000000000002', '52000000-0000-4000-8000-000000000001', '50000000-0000-4000-8000
-000000000001',
  '51000000-0000-4000-8000-000000000002', 0, 'NOT_DISPENSED', 'Out of stock - patient advised to buy
outside')
ON CONFLICT (id) DO NOTHING;

-- ---------- consultation 2 (today, pending in pharmacist queue) ----------
INSERT INTO consultations (id, patient_id, doctor_id, queue_entry_id, consultation_at, presenting_complain
t, clinical_notes)
VALUES ('40000000-0000-4000-8000-000000000002', '20000000-0000-4000-8000-000000000001',
        '00000000-0000-4000-8000-000000000002', '31000000-0000-4000-8000-000000000001', now() - INTERVAL '
95 minutes',
        'Review: raised blood sugar on home testing', 'FBS 168 mg/dl (patient report). Lifestyle advice
given.')
ON CONFLICT (id) DO NOTHING;
INSERT INTO consultation_diagnoses (id, consultation_id, line_number, condition_text, icd10_code) VALUES
 ('41000000-0000-4000-8000-000000000002', '40000000-0000-4000-8000-000000000002', 1, 'Type 2 diabetes
mellitus', 'E11.9'),
 ('41000000-0000-4000-8000-000000000003', '40000000-0000-4000-8000-000000000002', 2, 'Essential
hypertension', 'I10')
ON CONFLICT (id) DO NOTHING;
INSERT INTO prescriptions (id, prescription_number, consultation_id, patient_id, status, notes, submitted_
at)
VALUES ('50000000-0000-4000-8000-000000000002', 2, '40000000-0000-4000-8000-000000000002',
        '20000000-0000-4000-8000-000000000001', 'PENDING', 'Patient allergic to penicillin', now() - INTER
VAL '94 minutes')
ON CONFLICT (id) DO NOTHING;
INSERT INTO prescription_items (id, prescription_id, line_number, medicine_id, medicine_name, strength, do
sage_form,
                                dose, frequency, duration_days, quantity, quantity_unit, instructions) VAL
UES
 ('51000000-0000-4000-8000-000000000003', '50000000-0000-4000-8000-000000000002', 1, '10000000-0000-4000-8
000-000000000005',
  'Metformin', '500 mg', 'TABLET', '1 tablet', 'Twice daily (BD)', 30, 60, 'tablets', 'With meals'),
 ('51000000-0000-4000-8000-000000000004', '50000000-0000-4000-8000-000000000002', 2, '10000000-0000-4000-8
000-000000000006',
  'Amlodipine', '5 mg', 'TABLET', '1 tablet', 'Once daily (OD) in the morning', 30, 30, 'tablets', NULL)
ON CONFLICT (id) DO NOTHING;

-- Keep the sequence ahead of the fixed seed numbers.
SELECT setval('prescription_number_seq', GREATEST((SELECT max(prescription_number) FROM prescriptions), 1)
);
```

10. Security architecture summary

Concern	Design
Transport	TLS at the reverse proxy; HSTS; internal network for backend and database
Identity	Local accounts, BCrypt(12), lockout, forced password change
Tokens	HS256 JWT 15 min (memory); rotating refresh token 12 h (HttpOnly, Secure, SameSite=Strict, path-scoped), hashed at rest, reuse detection
Authorisation	Role checks per endpoint (`@PreAuthorize`), ownership checks in services, dedicated DTOs per audience
Data protection	No PHI in logs/browser storage; `Cache-Control: no-store`; public DTO whitelist; database volume on encrypted disk recommended; encrypted backups
Integrity	DB constraints, append-only audit trigger, optimistic and pessimistic locking
Abuse	Rate limiting (login, public), request size limit, generic auth errors
Accountability	Audit events for writes, sensitive reads and denials

11. Operations

- **Health:** `/actuator/health/liveness` (process), `/actuator/health/readiness` (includes database). Docker healthchecks and the uptime probe use readiness.
- **Logging:** JSON (ECS) to stdout, collected by Docker; fields `@timestamp`, `log.level`, `message`, `trace.id` = request id, `user.id`. No PHI (NFR-010).
- **Metrics:** Micrometer + Prometheus registry at `/actuator/prometheus` on the application port, restricted to private-network callers and not routed by the proxy.
- **Backups:** `pg_basebackup` nightly + continuous WAL archiving (e.g. pgBackRest or WAL-G) to encrypted off-site storage; quarterly restore drill restoring into a fresh container and running the smoke test.
- **Housekeeping:** the `retention` jobs run nightly at 00:05 (`APP_TIMEZONE`): auto-close past sessions, anonymise old queue PII, purge expired refresh tokens and idempotency keys; each run writes `RETENTION_JOB_EXECUTED`.

12. Scalability and evolution

- The backend is stateless; horizontal scaling needs only a load balancer. Rate-limit counters are per instance (acceptable; a shared store can be added later).
- Database growth (section 3.2) is small for PostgreSQL; audit events may be partitioned by year when they exceed ~50 million rows.
- Module boundaries allow extracting e.g. the queue into its own service later, keeping the same API paths.
- The API is versioned (`/api/v1`); breaking changes would introduce `/api/v2` alongside.